A Handbook of Graphs and Diagrams in Group Theory and Topology

Detailed constructions, reconstructions, and TikZ drawings

Hanwen Shen
Stevens Institute of Technology

December 2025

Abstract

This handbook collects a unified "diagrammatic toolkit" for combinatorial group theory and low-dimensional topology. Its goal is practical: to make the standard graphs, diagrams, and 2-complexes that encode algebraic statements easy to construct, compare, and use as algorithmic certificates. For each object, we give (i) a precise definition in a consistent combinatorial model (words, labeled graphs, finite 2-dimensional CW- and polygonal complexes); (ii) an existence/uniqueness statement clarifying what is canonical (up to label- and basepoint-preserving isomorphism) and what depends on choices; (iii) **Algorithm A** (Forward) turning algebraic input into a graph/diagram/complex; (iv) **Algorithm B** (Inverse) extracting algebraic data or a certificate back from the combinatorial object; and (v) worked examples drawn in TikZ, with explicit step-by-step construction and reconstruction.

Contents

1 Introduction

This handbook collect standard graphs and diagrams used in combinatorial group theory and low-dimensional topology. **Please read textbook for formal study.**

For each object we include:

- a definition;
- an existence/uniqueness theorem (what is canonical, and what is not);
- **Algorithm A (Forward)**: algebra/input → graph/diagram/complex;
- **Algorithm B (Inverse)**: graph/diagram/complex → algebraic data/certificate;
- **worked examples with step-by-step construction and reconstruction, each drawn in TikZ**.

Important meta-convention. When we say "unique up to isomorphism" we always mean *label-preserving isomorphism* (and basepoint-preserving if a basepoint is part of the data).

2 Conventions

2.1 Words, reductions, and lengths

Fix a finite alphabet of generators $X = \{x_1, \dots, x_n\}$ and let $X^{\pm 1} = X \cup X^{-1}$. A *word* is a finite string $w = x_{i_1}^{\varepsilon_1} \cdots x_{i_k}^{\varepsilon_k}$ with $\varepsilon_j \in \{\pm 1\}$. Its *length* is $|w| = k$.

A word is *freely reduced* if it has no adjacent cancellation xx^{-1} or $x^{-1}x$. **Convention (terminology):** throughout these notes, *reduced* means *freely reduced*. We write $\mathrm{red}(w)$ for the freely reduced form of w (obtained by repeatedly cancelling adjacent inverse pairs).

A reduced word w is *cyclically reduced* if every cyclic rotation of w is reduced, equivalently the first letter is not inverse to the last letter.

Let $X = \{a, b\}$ and $X^{\pm 1} = \{a^{\pm 1}, b^{\pm 1}\}$.

Words (what is / what is not).

- **Is a word:** $w = a\,b^{-1}\,a\,a^{-1}\,b$ (a finite string of letters in $X^{\pm 1}$).
- **Is a word:** $w = (a^{-1})^3\,b\,a^2$ (this is just shorthand for $a^{-1}a^{-1}a^{-1}baa$).
- **Not a word (in this model):** $a^{1/2}b$ (exponents other than ± 1 are not letters of $X^{\pm 1}$).
- **Not a word (as a finite string):** $ababab\cdots$ (an infinite string; words here are finite).

Free reduction: reduced vs. not reduced. Recall: *reduced* means *freely reduced*.

- **Not reduced (has adjacent cancellation):**

  $$w_1 = a\,a^{-1}\,b.$$

  Here aa^{-1} cancels, so $\mathrm{red}(w_1) = b$ and $|w_1| = 3$, $|\mathrm{red}(w_1)| = 1$.

- **Not reduced (multiple cancellations):**

  $$w_2 = a\,b\,b^{-1}\,a^{-1}.$$

  Cancel bb^{-1} to get aa^{-1}, so $\mathrm{red}(w_2) = \varepsilon$ (the empty word).

- **Reduced (no adjacent inverse pairs):**

  $$w_3 = a\,b\,a^{-1}\,b^{-1}.$$

  No adjacent letters are inverse, hence $\mathrm{red}(w_3) = w_3$.

- **Reduced (even though inverse letters appear non-adjacently):**

  $$w_4 = a\,b\,a^{-1}.$$

  This is reduced because the a and a^{-1} are not adjacent, so there is no free cancellation.

Cyclic reduction: cyclically reduced vs. not cyclically reduced. A reduced word is *cyclically reduced* iff its first letter is not the inverse of its last letter.

- **Reduced but not cyclically reduced:**

  $$w_5 = a\,b\,a^{-1}.$$

  It is reduced, but the first letter a is inverse to the last letter a^{-1}, so it is *not* cyclically reduced. Indeed, the cyclic rotation $b\,a^{-1}\,a$ is not reduced (it contains $a^{-1}a$).

- **Cyclically reduced:**
$$w_6 = a\,b\,a^{-1}\,b^{-1}.$$
This is reduced and the first letter a is not inverse to the last letter b^{-1}, hence it is cyclically reduced.

- **Cyclically reduced (short words):**
$$w_7 = a, \qquad w_8 = b^{-1}.$$
Any reduced word of length 1 is cyclically reduced.

- **Not cyclically reduced (after reducing first):**
$$w_9 = a\,b\,b^{-1}\,a^{-1}.$$
This word is not reduced. After free reduction, $\mathrm{red}(w_9) = \varepsilon$. (If you only apply the cyclically reduced definition to *reduced* words, you first reduce and then check.)

2.2 Complexity parameters (what "input size" means)

For algorithmic statements we will use the following standard size parameters:

- For words: length $|w|$.
- For a subgroup input $H = \langle w_1, \dots, w_m \rangle \leq F(X)$: total input length
$$L = \sum_{i=1}^{m} |w_i| \quad \text{(after free reduction)}.$$
- For a finite graph Γ: number of vertices $|V|$, edges $|E|$.
- For a finite 2-complex/diagram D: numbers $(|V|, |E|, |F|)$, and total perimeter
$$P = \sum_{f \in \mathrm{Faces}(D)} |\partial f|.$$
- For a finite presentation $\mathcal{P} = \langle X \mid R \rangle$: total relator length
$$\|R\| = \sum_{r \in R} |r|.$$

When groups are infinite, one usually works with *implicit* representations (normal forms, rewriting systems, coset enumeration, etc.); complexity then depends on the chosen representation. We will state complexities in terms of the combinatorial objects actually built (numbers of vertices/edges/faces) and the lengths of words manipulated.

Binary vs. unary encoding for integers

An integer $N \geq 0$ may be provided in:

- **Binary encoding:** input size $\langle N \rangle := \lfloor \log_2(N) \rfloor + 1$ bits (for $N \geq 1$; and $\langle 0 \rangle := 1$).
- **Unary encoding:** input size $\Theta(N)$ symbols (rare in group theory, but sometimes used to model "pseudopolynomial" behavior).

Words and presentations: typical examples

- **Word problem input:** a word $w \in (X^{\pm 1})^*$ with size $|w|$.
- **Subgroup membership input:** $H = \langle w_1, \dots, w_m \rangle \leq F(X)$ together with a query word w. The total size is typically $L + |w|$, where $L = \sum_i |w_i|$ after free reduction.
- **Finite presentation input:** $\mathcal{P} = \langle X \mid R \rangle$ with size

$$|X| + \|R\|, \qquad \|R\| := \sum_{r \in R} |r|$$

(again with relators freely reduced; one may also include the number of relators $|R|$ separately).

Numeric labels and weights on graphs/complexes

Often a graph or 2–complex comes with additional integer data (lengths, weights, exponents, capacities).

- **Edge/vertex weights in binary:** if each edge $e \in E$ has an integer weight $w(e) \in \mathbb{Z}$ given in binary, the input size includes

$$\sum_{e \in E} \langle |w(e)| \rangle$$

(and similarly for vertex weights).

- **Geometric coordinates:** if vertices are given by rational coordinates in $\mathbb{Q}^d$, write each coordinate as p/q in lowest terms; the input size includes the total bit-length of all numerators/denominators:

$$\sum_{\text{coords}} \big(\langle |p| \rangle + \langle |q| \rangle \big).$$

Matrices and linear representations (common in algorithmic group theory)

- **Matrix group input:** generators $A_1, \dots, A_m \in \mathrm{GL}_d(\mathbb{Z})$ (or $\mathrm{GL}_d(\mathbb{Q})$). The input size is

$$m \cdot d^2 \; + \; \sum_{i=1}^{m} \sum_{j,k} \langle |(A_i)_{jk}| \rangle$$

for integer entries (or add numerator/denominator bit-lengths for rational entries).

- **Word in matrix generators:** a word of length n in the alphabet $\{A_i^{\pm 1}\}$ has size n in the word metric, but the intermediate matrix entries may grow; complexity bounds must specify whether arithmetic is counted in *bit complexity*.

Automata and rewriting systems (implicit representations)

When groups or languages are described implicitly, the "size" is the description length of the device.

- **Finite automaton input:** a DFA/NFA with $|Q|$ states over alphabet Σ. A standard encoding size is

$$O\big(|Q| \cdot |\Sigma|\big)$$

for the transition table (plus the size of the accept set).

- **Rewriting system input:** a finite set of rewrite rules $\ell \to r$. A natural size parameter is the total rule length
$$\|\mathcal{R}\| := \sum_{(\ell \to r) \in \mathcal{R}} \big(|\ell| + |r|\big).$$

- **Straight-line program (SLP) / compressed word:** a context-free grammar producing a single word. The input size is the number of productions (and total symbol count), which can be exponentially smaller than the expanded word length. When inputs may be compressed, one must state whether complexity is measured in terms of SLP size or expanded length.

Common "input packages" (worked-size examples)

- **Van Kampen diagram input:** a finite planar 2–complex D with labels on directed edges. Size can be taken as $(|V|, |E|, |F|)$ together with the total perimeter $P = \sum_f |\partial f|$. If labels are words over $X^{\pm 1}$, add their total length as well.

- **Graph-of-groups / Bass–Serre input:** a finite graph Γ plus finitely presented vertex/edge groups and attaching maps. A practical size measure is $|V(\Gamma)| + |E(\Gamma)|$ plus the presentation sizes of all vertex/edge groups and the word-lengths of images of edge generators under the attaching maps.

- **Surface input:** a triangulated surface given by its face list (triples of vertex indices). Size is $\Theta(|F|)$ plus the bit-length of indices if vertices are named in binary.

Bit complexity vs. unit-cost arithmetic

When algorithms involve integer arithmetic (e.g. matrix multiplication, coordinate computations), it is important to specify the cost model:

- **Unit-cost RAM model:** each arithmetic operation on machine words costs $O(1)$ (good for fixed-size integers).

- **Bit complexity model:** the cost of arithmetic depends on operand bit-length (required for honest bounds when numbers can grow).

Unless stated otherwise, we count *bit complexity* whenever integer sizes may grow during the computation.

Remark. The same mathematical object can have very different input sizes depending on representation (e.g. an element given by an explicit word vs. an SLP-compressed word, or a subgroup given by a generating set vs. a Stallings graph). Accordingly, complexity statements should always specify the representation and which size parameters are being used.

2.3 X-labeled graphs and path labels

An *X-labeled oriented graph* is a directed graph whose directed edges are labeled by $X^{\pm 1}$, with the rule: reversing an edge inverts its label. For a directed path $p = e_1 \cdots e_k$, define $\mathrm{lab}(p) = \mathrm{lab}(e_1) \cdots \mathrm{lab}(e_k)$.

Terminology consistency: when we say *labeled graph* in these notes, we always mean *X-labeled* unless explicitly stated otherwise.

A labeled graph is *folded (deterministic)* if for every vertex v and every label $a \in X^{\pm 1}$ there is at most one outgoing edge from v with label a. Equivalently, reading a reduced word from a vertex determines at most one reduced path.

2.4 CW-complexes and polygonal complexes

A *CW-complex* is a space built by attaching k-cells (copies of open disks) inductively via continuous attaching maps on their boundaries. In these notes we only use *finite 2-dimensional* CW-complexes (0-, 1-, and 2-cells).[10]

A *polygonal complex* is a 2-dimensional CW-complex in which each 2-cell is attached along the boundary cycle of a polygon (an n-gon subdivided into n oriented edges), with attaching maps that send edges to edge-paths. Thus:

$$\text{polygonal complex} \subseteq \text{(2-dimensional CW-complex)}.$$

Presentation complexes and van Kampen diagrams are polygonal complexes.

Examples: CW-complexes and polygonal complexes (simple)

CW-complexes (finite, low-dimensional examples).

- **A finite graph.** Any finite graph Γ is a 1–dimensional CW-complex: vertices are 0–cells and edges are 1–cells attached at their endpoints.
- **The circle S^1.** One 0–cell v and one 1–cell e attached with both endpoints mapped to v.
- **A disk D^2.** One 0–cell, one 1–cell forming the boundary circle, and one 2–cell attached along that boundary loop.
- **The sphere S^2.** One 0–cell and one 2–cell attached by the constant map $S^1 \to$ (the 0–cell). (Equivalently, two 2–cells glued along their boundary circle.)

Polygonal complexes (2D CW-complexes with polygonal faces).

- **A single polygon.** Take an n–gon and attach its boundary edges in a cycle; this is the simplest polygonal complex with one face.
- **The torus T^2 from one square.** One vertex, two edges labeled a, b, and one square 2–cell attached by the boundary word
$$aba^{-1}b^{-1}.$$
This is a polygonal complex with one 4–gonal face.
- **A presentation complex (one-relator case).** For $\mathcal{P} = \langle X \mid r \rangle$, take a wedge of $|X|$ circles (one for each generator) and attach one 2–cell along the loop labeled by r. This is a polygonal complex whose single face has perimeter $|r|$.
- **A van Kampen diagram.** A van Kampen diagram is a finite planar polygonal complex (topologically a disk) whose faces are labeled by relators (and inverses), with a distinguished outer boundary cycle labeling a word equal to 1 in the group.

One simple non-example (CW but not polygonal).

- **A non-combinatorial attachment.** Attach a 2–cell to a graph by a continuous map $S^1 \to \Gamma$ that is not a combinatorial edge-loop (e.g. it collapses an arc of S^1 to a point). The result is still a 2–dimensional CW-complex, but it is not polygonal under the combinatorial definition above.

What is *not* a CW-complex? (standard non-examples)

(1) Failure of closure-finiteness: the infinite wedge (Hawaiian earring type). Let $H \subset \mathbb{R}^2$ be the *Hawaiian earring*, the union of circles

$$H = \bigcup_{n \geq 1} C_n, \qquad C_n = \{(x, y) : (x - 1/n)^2 + y^2 = (1/n)^2\},$$

all tangent at the origin. This space is *not* a CW-complex: any cell decomposition would force the point of tangency to lie in the closure of infinitely many 1–cells, violating the closure-finite condition.

(2) Failure of weak topology: spaces with a "bad" quotient topology. There are classical examples of cell-like decompositions or quotient spaces where the resulting topology is not the weak topology coming from cells. Such spaces can be built by gluing disks in a purely set-theoretic way, but the topology is too coarse/fine to satisfy axiom (W), hence the space is not CW. (Concrete examples can be found in standard topology texts under "weak topology fails for CW".)

(3) Non-Hausdorff spaces (CW-complexes are Hausdorff). Every CW-complex is Hausdorff. Hence any non-Hausdorff space is automatically *not* a CW-complex. For example, the *line with two origins* is not Hausdorff, so it is not a CW-complex.

(4) The long line (not a CW-complex). The (closed) long line is a connected 1–manifold that is not second countable and has pathological compactness properties. It is not homotopy equivalent to any CW-complex and, in particular, is not a CW-complex.

Remark. Most "reasonable" spaces encountered in algebraic topology (manifolds, polyhedra, simplicial complexes, finite graphs, finite 2–complexes, etc.) are CW-complexes. Non-examples typically arise from infinite accumulation phenomena (e.g. infinitely many cells piling up at a point) or from pathological topologies.

2.5 Simply connected (precise meaning)

A space Y is *simply connected* if it is path-connected and every loop $\gamma : S^1 \to Y$ is null-homotopic. Equivalently, $\pi_1(Y)$ is trivial. For a finite 2-complex/diagram, "simply connected" means its underlying topological space has trivial fundamental group. In van Kampen diagrams, simply connectedness ensures the diagram fills a single boundary component (a disk) and supports van Kampen's lemma.[10, 24]

2.6 CAT(0)

A *CAT(0) space* is a geodesic metric space in which geodesic triangles are *no thicker* than comparison triangles in Euclidean space. Intuitively: the space has global nonpositive curvature; geodesics are unique and convexity properties hold.

In these notes CAT(0) appears for *cube complexes* (Salvetti complexes and their universal covers). A standard fact (Gromov's link condition) says: a finite-dimensional cube complex is nonpositively curved (and its universal cover is CAT(0)) if and only if the link of every vertex is a *flag* simplicial complex. For the Salvetti complex of a RAAG, the link is flag by construction, so the universal cover is CAT(0).[1, 9]

2.7 Presentations and the presentation 2-complex

A group presentation is $\mathcal{P} = \langle X \mid R \rangle$ with R a set of relators (words in $X^{\pm 1}$). The associated *presentation 2-complex* $K(\mathcal{P})$ has: one vertex, one oriented 1-cell for each $x \in X$, and one 2-cell for each $r \in R$ attached along a loop labeled by r.

Examples: finite presentation, finite relator set, finitely generated but not finitely presented, coherent vs. not coherent

(A) Finite presentations ("finitely presented groups").

- **Free group of rank n.**
$$F_n \;=\; \langle x_1, \ldots, x_n \mid \; \rangle$$
(empty relator set).

- **Free abelian group $\mathbb{Z}^n$.**
$$\mathbb{Z}^n \;=\; \big\langle x_1, \ldots, x_n \;\big|\; [x_i, x_j] = 1 \text{ for all } 1 \le i < j \le n \big\rangle,$$
where $[x_i, x_j] = x_i x_j x_i^{-1} x_j^{-1}$.

- **Cyclic group of order m.**
$$C_m \;=\; \langle a \mid a^m = 1 \rangle.$$

- **Surface group of genus $g \ge 1$.**
$$\pi_1(\Sigma_g) \;=\; \left\langle a_1, b_1, \ldots, a_g, b_g \;\middle|\; \prod_{i=1}^{g} [a_i, b_i] = 1 \right\rangle.$$

(B) Finite relator set but *infinite* presentation (infinitely many generators). A presentation can have finitely many relators and still be infinite if X is infinite. For example, the free group of countably infinite rank has the presentation
$$F_\infty \;=\; \langle x_1, x_2, x_3, \cdots \mid \; \rangle,$$
which has *zero* relators but infinitely many generators.

(C) A finitely generated group that is *not* finitely presented. A classical example is the lamplighter group

$$L \;=\; (\mathbb{Z}/2\mathbb{Z}) \wr \mathbb{Z} \;\cong\; \Big\langle a, t \;\Big|\; a^2 = 1,\; [a, t^n a t^{-n}] = 1 \text{ for all } n \in \mathbb{Z} \Big\rangle .$$

It is finitely generated (by a, t) but requires infinitely many independent commutation relations, hence it is not finitely presented.

(D) Coherent groups (definition + basic examples). A group G is *coherent* if every finitely generated subgroup $H \leq G$ is finitely presented.

- **Free groups are coherent:** every finitely generated subgroup of a free group is free (Nielsen–Schreier), hence finitely presented.
- **Surface groups are coherent:** finitely generated subgroups of $\pi_1(\Sigma_g)$ are again free or surface groups, hence finitely presented.
- **Finitely generated abelian groups are coherent:** subgroups of $\mathbb{Z}^n$ are free abelian of finite rank.

(E) Non-coherent groups (a standard example). A standard source of non-coherence is $F_2 \times F_2$. It contains finitely generated subgroups that are *not* finitely presented, hence $F_2 \times F_2$ is not coherent. (Equivalently: there exists a finitely generated subgroup $H \leq F_2 \times F_2$ with no finite presentation.)

(F) A finitely generated subgroup with an *infinite* presentation (inside a finitely presented group). The previous item can be rephrased as: inside the finitely presented group $F_2 \times F_2$, there exists a finitely generated subgroup H that is not finitely presented; in particular, any presentation of H must use infinitely many relators (or infinitely many generators).

Remark (finite vs. finitely presented). A presentation $\langle X \mid R \rangle$ is called *finite* if both X and R are finite. A group is *finitely presented* if it admits *some* finite presentation. A group can be finitely generated but not finitely presented (as in (C)), and a group can have a presentation with finitely many relators but still not be finitely presented if it uses infinitely many generators (as in (B)).

2.8 Isomorphisms and non-uniqueness (what is canonical and what is not)

Two based X-labeled graphs $(\Gamma, *)$ and $(\Gamma', *')$ are *isomorphic* if there is a graph isomorphism preserving the basepoint and all labels. If you additionally label vertices by chosen *representative words* (e.g. coset representatives), then that extra labeling is *not canonical*: changing representatives changes vertex names but usually only by a label-preserving graph isomorphism.

Diagrams over a fixed complex are isomorphic if they are cellularly isomorphic and the maps to the target complex agree. For disk/annular/van Kampen diagrams: existence is often equivalent to an algebraic statement (word triviality, conjugacy), but **uniqueness typically fails**: the same word can have many non-isomorphic diagrams. What is often canonical is an *invariant* extracted from diagrams (e.g. minimal area), not the diagram itself.

2.9 Boundary

We model diagrams as finite, connected, 2–dimensional CW–complexes (often called *combinatorial* 2–complexes) whose 1–skeleton is a graph and whose 2–cells are attached by combinatorial edge-loops.

Boundary of a disk diagram. Let D be a *disk diagram*, i.e. a finite, contractible, planar 2–complex together with an embedding of its underlying space into the plane such that its complement has exactly one unbounded component. The *outer boundary* of D, denoted ∂D, is the (topological) frontier of D in the plane; combinatorially it is realized by a closed edge-path in the 1–skeleton.

A convenient purely combinatorial description is in terms of *degrees of edges*. For an edge e in $D^{(1)}$, define

$$\deg(e) \;:=\; \#\{\text{2–cells incident to } e \text{ (counted with multiplicity)}\}.$$

Then the *boundary subcomplex* $\partial D \subset D^{(1)}$ is the 1–dimensional subcomplex consisting of all vertices and edges with $\deg(e) = 1$. Each connected component of ∂D is a cycle; for a disk diagram there is exactly one such component, the outer boundary.

Boundary of a face and perimeter. For a 2–cell (face) f of D, the attaching map of f is a combinatorial map

$$\phi_f : S^1 \longrightarrow D^{(1)}$$

whose image is a closed edge-path. We denote this closed edge-path by ∂f and call it the *boundary cycle* of f. Its *perimeter* is the length of this edge-path (counting edge occurrences with multiplicity):

$$|\partial f| \;:=\; \ell(\partial f).$$

Equivalently, if one works in cellular chain groups $C_2(D;\mathbb{Z}) \to C_1(D;\mathbb{Z})$ with a choice of orientations, the cellular boundary operator sends

$$\partial_2(f) \;=\; \sum_{e \in D^{(1)}} \epsilon_{f,e}\, e,$$

and $|\partial f|$ is the ℓ^1–norm of $\partial_2(f)$ when each traversal of an edge is counted (so repeated edges contribute repeatedly).

Boundary word. If the edges of $D^{(1)}$ are labeled (e.g. by generators) and oriented, then choosing an orientation of ∂D and a basepoint on ∂D yields a *boundary word* $w(\partial D)$ obtained by reading labels along ∂D. Changing the basepoint changes $w(\partial D)$ by a cyclic permutation; reversing the orientation inverts the word. Thus, canonically, the boundary word of a disk diagram is defined as a *cyclic word* up to inversion.

2.10 Tilings (tessellations) and related cell structures

We use the term *tiling* (or *tessellation*) in a broad sense that covers the classical Euclidean picture as well as the CW–/combinatorial structures used throughout these notes (polygonal complexes, triangulations, and cubical complexes).

Definition 2.1 (Tiling / tessellation)**.** Let X be a topological space (typically $X = \mathbb{R}^n$ or a surface). A *tiling* of X is a collection of subsets (tiles)

$$\mathcal{T} = \{T_i\}_{i \in I}, \qquad T_i \subseteq X,$$

such that:

1. (**Cover**) $X = \bigcup_{i \in I} T_i$.
2. (**No overlap in the interior**) For $i \neq j$,

$$\operatorname{int}(T_i) \cap \operatorname{int}(T_j) = \varnothing.$$

A tiling is called *locally finite* if every compact $K \subseteq X$ meets only finitely many tiles.

Definition 2.2 (Face-to-face tiling (often assumed))**.** A locally finite tiling $\mathcal{T}$ is *face-to-face* if for any $i \neq j$, the intersection $T_i \cap T_j$ is either empty or a (possibly lower-dimensional) *common face* of both T_i and T_j (e.g. a common vertex/edge/face in the polyhedral setting).

Definition 2.3 (Polygonal tiling / polygonal complex (2D))**.** A *polygonal tiling* of a surface (or planar region) is a face-to-face tiling by polygons. Equivalently (combinatorial viewpoint), it is a 2–dimensional CW–complex in which every 2–cell is attached along the boundary cycle of an n–gon by a combinatorial attaching map.

Definition 2.4 (Triangulation (simplicial tiling) in dimension 2)**.** A *triangulation* of a surface (or planar region) is a polygonal tiling in which every 2–cell is a triangle. Equivalently, it is a 2–dimensional simplicial complex whose underlying space is the given surface/region. A *subdivision* is a refinement obtained by subdividing faces into smaller faces (e.g. by a specified rule).

Definition 2.5 (Square tiling of $\mathbb{R}^2$)**.** The *square tiling* of $\mathbb{R}^2$ is the face-to-face tiling by unit squares with vertices in $\mathbb{Z}^2$:

$$\mathcal{T}_{\square} = \big\{ [m, m+1] \times [n, n+1] \subset \mathbb{R}^2 \ : \ m, n \in \mathbb{Z} \big\}.$$

Its 1–skeleton is the integer grid graph, and its 2–cells are the unit squares.

Definition 2.6 (Cubical tiling / cube complex)**.** A *cube complex* is a CW–complex obtained by gluing Euclidean cubes $[0,1]^k$ along faces via isometries, with attaching maps that are combinatorial (face-to-face). A *cubical tiling* of a space X is a locally finite cube complex structure on X in which the cubes form a tiling in the sense of Definition above (covering and disjoint interiors).

Remark 2.7 (How these notions are used here).

- Polygonal complexes generalize planar polygonal tilings and serve as the ambient objects for disk diagrams and presentation complexes.
- Triangulations are used when one wants to replace a polygonal diagram by a triangular one without changing the boundary word.
- Cubical complexes (e.g. Salvetti complexes) are built from squares and higher-dimensional cubes; their universal covers can be viewed as (locally finite) cubical tilings in the metric/CAT(0) setting.

3 Cayley graph

Definition

Let G be a group generated by a set S. The *Cayley graph* $\mathrm{Cay}(G, S)$ has vertex set G and, for each $g \in G$ and $s \in S$, a directed labeled edge

$$g \xrightarrow{s} gs.$$

If $S = S^{-1}$ one often views it as an undirected labeled graph.[2]

Existence/uniqueness (precise statement)

Theorem 3.1 (Cayley graph: existence and uniqueness for marked groups)**.** *Let (G, S) be a* marked group*, meaning S is a specified generating set and edge labels are the symbols in S.*

1. *(**Existence**) $\mathrm{Cay}(G, S)$ exists and is connected.*
2. *(**Uniqueness**) If $\varphi : (G, S) \to (G', S')$ is an isomorphism of marked groups (a group isomorphism with $\varphi(S) = S'$ and preserving labels), then $\Phi(g) = \varphi(g)$ defines a unique based label-preserving isomorphism*

$$\Phi : \mathrm{Cay}(G, S) \longrightarrow \mathrm{Cay}(G', S').$$

3. *(**Reconstruction**) Conversely, if $\Phi : \mathrm{Cay}(G, S) \to \mathrm{Cay}(G', S')$ is a based label-preserving graph isomorphism sending 1_G to $1_{G'}$, then there exists a unique marked-group isomorphism $\varphi : (G, S) \to (G', S')$ such that $\Phi(g) = \varphi(g)$ for all $g \in G$.*

Algorithm A (Forward): construct $\mathrm{Cay}(G, S)$

[2, 4]

Input. A group G, a generating set S, and procedures:

- multiply: $(g, s) \mapsto gs$,
- invert: $s \mapsto s^{-1}$,
- equality test or normal forms (needed to avoid duplicating vertices when exploring).

Output. A graph object; for infinite groups, usually given as an *adjacency oracle*:

$$\mathrm{Adj}(g, s) = gs.$$

Steps (conceptual construction).

1. Create one vertex v_g for each element $g \in G$.
2. For each $g \in G$ and each $s \in S$, add the directed edge $v_g \xrightarrow{s} v_{gs}$.

Steps (finite window: BFS ball of radius N). To *draw* a finite portion, compute the ball $B_N(1_G)$ in the word metric:

1. Initialize a queue with the identity element 1_G at distance 0.
2. Maintain a dictionary `seen` from group elements to their discovered vertex.
3. While the queue is nonempty:
   (a) pop an element g with its distance d,
   (b) if $d = N$, do not expand further,
   (c) otherwise for each $s \in S$: compute $h = gs$; if h is unseen, add it with distance $d + 1$; record the edge $g \xrightarrow{s} h$.
4. Output the induced subgraph on discovered vertices.

Boundary cases and implementation notes.

- **Generating set must generate.** If S does not generate G, then the Cayley *graph as defined above* has one connected component per right coset of $\langle S \rangle$ in G. In this handbook we usually assume S generates G, so $\mathrm{Cay}(G, S)$ is connected.
- **Trivial group.** If $G = \{1\}$, then there is exactly one vertex. Every $s \in S$ represents 1 and gives a loop edge; if you want a reduced picture you may omit such identity loops.
- **Identity in S.** If $1 \in S$, then every vertex has a loop labeled 1; this does not change word metric/geodesics and is usually ignored.

Complexity (rough, for the portion you actually build). [4]

- **Finite group, explicit build.** If $|G| < \infty$ and group operations are $O(1)$, then building the full Cayley graph takes $O(|G||S|)$ time and stores $O(|G||S|)$ directed edges.
- **BFS ball of radius N.** Let $M = |B_N(1_G)|$ be the number of discovered vertices. The BFS explores at most $M|S|$ edges, so time is $O(M|S|)$ group multiplications plus dictionary lookups; space is $O(M|S|)$ to store edges (or $O(M)$ if you store adjacency implicitly).

Algorithm B (Inverse): reconstruct a group from a Cayley-type graph

[2, 4]

Input. A connected X-labeled graph Γ with a base vertex e satisfying:

- (**Completeness**) for each vertex v and each $x \in X$, there is *exactly one* outgoing edge from v labeled x;
- (**Inverse edges**) reversing an x-edge gives an x^{-1}-edge.

These are precisely the local axioms satisfied by a Cayley graph.

Output. A group G presented as a quotient $F(X)/N$ such that $\Gamma \cong \mathrm{Cay}(G, X)$ as based labeled graphs.

Steps.

1. **Endpoint map.** For any word $w \in (X^{\pm 1})^*$, define $\mathrm{End}(w)$ to be the endpoint of the (unique) path labeled w starting at e.
2. **Define the relation subgroup.** Let
$$N = \{\, w \in F(X) \mid \mathrm{End}(w) = e \,\}.$$
(Equivalently: N is the set of reduced words labeling loops at e.)
3. **Check normality.** Because Γ is vertex-transitive by label-completeness, translating a loop by a path shows N is closed under conjugation in $F(X)$, hence $N \trianglelefteq F(X)$.
4. **Define the reconstructed group.** Set $G := F(X)/N$.
5. **Identify vertices with cosets.** Map $[w] \in G$ to the vertex $\mathrm{End}(w)$. This is well-defined because if $[w] = [w']$ then $ww'^{-1} \in N$ so $\mathrm{End}(w) = \mathrm{End}(w')$.
6. **Verify Cayley property.** The x-edge from $\mathrm{End}(w)$ goes to $\mathrm{End}(wx)$, matching right multiplication in G. Thus $\Gamma \cong \mathrm{Cay}(G, X)$.

Boundary cases and interpretation notes.

- **If completeness fails.** If Γ is *not* complete (some vertex has no outgoing x-edge), then $\mathrm{End}(w)$ is not defined for all words and the construction recovers only a *partial action* (a deterministic automaton), not a Cayley graph of a group.
- **If Γ is finite.** Then the reconstructed group G is finite and $|G| = |V(\Gamma)|$.
- **If Γ is infinite.** The definition $G = F(X)/N$ still makes sense abstractly, but computing a finite presentation for N from an arbitrary infinite Γ may be impossible without extra structure (for example, periodicity or a known presentation).

Complexity (rough). [4]

- Let Γ be finite with $|V|$ vertices and alphabet size $|X| = n$. Checking completeness/inverse-edge conditions costs $O(|V|n)$ by scanning outgoing edges.
- Constructing the multiplication table (if desired) costs $O(|V|^2 n)$ naively: for each pair of vertices you multiply by reading a word that sends one to the other. If instead you only keep the Cayley graph structure (adjacency by generators), then storing the group operation is unnecessary and the graph itself already gives right-multiplication in $O(1)$ per generator step.

Example 1: $\mathbb{Z} = \langle t \rangle$ (a line)

Forward construction (Algorithm A).

1. Choose normal form: each element of $\mathbb{Z}$ is an integer k.
2. Vertices: one vertex for each $k \in \mathbb{Z}$.
3. For each k, compute $k + 1$ and add the edge $k \xrightarrow{t} k + 1$.
4. Reverse edges are labeled t^{-1}.

Inverse reconstruction (Algorithm B). Assume you are given an infinite line with edges labeled t to the right.

1. Choose basepoint $e = 0$.
2. For the word t^m (reduced), the t^m-path from 0 ends at vertex m. So $\mathrm{End}(t^m) = m$.
3. A word labels a loop at 0 iff its endpoint is 0, which occurs only for the empty word. Thus $N = \{1\}$.
4. Hence $G = F(t)/N \cong F(t) \cong \mathbb{Z}$, and the graph is the Cayley graph of $\mathbb{Z}$.

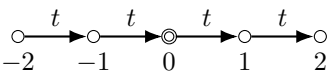

Figure 1: A finite window in $\mathrm{Cay}(\mathbb{Z}, \{t\})$. Reverse edges carry label t^{-1}.

Example 2: $\mathbb{Z}^2 = \langle a, b \mid [a, b] \rangle$ (a grid)

Forward construction (Algorithm A).

1. Choose normal form: elements are pairs $(i, j) \in \mathbb{Z}^2$.
2. Vertices: one vertex for each (i, j).
3. For each (i, j), add the edge $(i, j) \xrightarrow{a} (i + 1, j)$.
4. For each (i, j), add the edge $(i, j) \xrightarrow{b} (i, j + 1)$.
5. Reverse edges are labeled a^{-1} and b^{-1}.

Inverse reconstruction (Algorithm B). Assume you are given the labeled grid and basepoint $e = (0, 0)$.

1. Compute $\mathrm{End}(ab)$: from $(0, 0)$ go right via a to $(1, 0)$, then up via b to $(1, 1)$. So $\mathrm{End}(ab) = (1, 1)$.
2. Compute $\mathrm{End}(ba)$: from $(0, 0)$ go up via b to $(0, 1)$, then right via a to $(1, 1)$. So $\mathrm{End}(ba) = (1, 1)$.
3. Therefore $\mathrm{End}(ab) = \mathrm{End}(ba)$, so $\mathrm{End}(aba^{-1}b^{-1}) = e$. Thus the commutator word labels a loop at e, hence $[a, b] \in N$.
4. The set of all square-boundary loops generate all loops in the grid, so $N = \langle\langle [a, b] \rangle\rangle$ and $G \cong \langle a, b \mid [a, b] \rangle \cong \mathbb{Z}^2$.

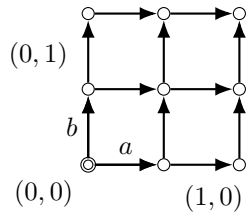

Figure 2: A finite patch of $\mathrm{Cay}(\mathbb{Z}^2, \{a, b\})$ (all horizontal edges labeled a, all vertical edges labeled b).

Example 3: $F_2 = \langle a, b \rangle$ (a tree)

Forward construction (Algorithm A).

1. Choose normal form: each element of $F(a, b)$ is a reduced word in $\{a^{\pm 1}, b^{\pm 1}\}$.
2. Vertices: all reduced words w.
3. For each vertex w and generator $x \in \{a^{\pm 1}, b^{\pm 1}\}$: compute $\mathrm{red}(wx)$ and add the edge $w \xrightarrow{x} \mathrm{red}(wx)$.
4. This produces a 4-regular tree because reduced words have unique reduced prefixes and no cycles occur.

Inverse reconstruction (Algorithm B).

1. In a tree, any reduced closed path would create a cycle, impossible. Hence the only reduced loop is trivial.
2. Therefore $N = \{1\}$ and $G \cong F(a, b)$ is free.

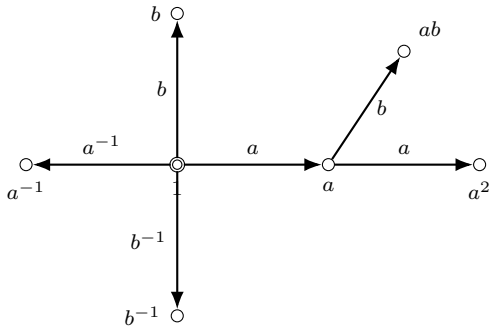

Figure 3: A small ball in $\mathrm{Cay}(F_2, \{a, b\})$. Extending produces an infinite 4-regular tree.

4 Subgroup graph (Stallings graph)

Definition

Let $F(X)$ be a free group on X and $H \leq F(X)$ a finitely generated subgroup. A *Stallings graph* for H is a finite connected based folded X-labeled graph $(\Gamma_H, *)$ such that reduced loops at $*$ label exactly the elements of H[20, 14]:

$$H = \{\mathrm{lab}(p) \in F(X) \mid p \text{ is a reduced loop at } * \text{ in } \Gamma_H\}.$$

Existence/uniqueness (precise statement)

Theorem 4.1 (Stallings graph: existence, algorithmic construction, and uniqueness)**.** *Fix a basis X of the free group $F(X)$ and let $H \leq F(X)$ be finitely generated.*

1. *(**Existence**) There exists a finite connected based folded X-labeled graph $(\Gamma_H, *)$ such that the reduced loop language at $*$ equals H.*
2. *(**Cover interpretation**) Let R_X be the rose with $|X|$ loops labeled by X. Then H corresponds to a based covering $p : \widetilde{R} \to R_X$ and Γ_H is the* finite core *of $\widetilde{R}$ (obtained by deleting all hanging trees not containing the basepoint).*
3. *(**Algorithmic construction**) Given any finite generating set $H = \langle w_1, \ldots, w_m \rangle$, the* folding + pruning *algorithm (Algorithm A below) terminates and outputs a graph isomorphic to $(\Gamma_H, *)$.*
4. *(**Uniqueness**) The isomorphism class of $(\Gamma_H, *)$ depends only on H (and the fixed basis X), not on the chosen generating set and not on the order in which folds are performed.*
5. *(**Consequences that are read off from** Γ_H)*
   - *Membership: $w \in H$ iff the reduced word w labels a loop at $*$ in Γ_H.*
   - *Rank:* $\mathrm{rank}(H) = |E(\Gamma_H)| - |V(\Gamma_H)| + 1$.
   - *Finite index: H has finite index in $F(X)$ iff $\Gamma_H \to R_X$ is a covering (equivalently: each vertex has exactly one outgoing edge labeled x for each $x \in X$). If so, $[F(X) : H] = |V(\Gamma_H)|$.*

Algorithm A (Forward): generators → Stallings graph (detailed)

[20, 14, 23]

Input. Freely reduced words $w_1, \ldots, w_m \in (X^{\pm 1})^*$ generating $H = \langle w_1, \ldots, w_m \rangle \leq F(X)$.

Output. The folded core graph $(\Gamma_H, *)$.

Data structure convention. We view Γ as:

- a set of vertices V with a distinguished basepoint $*$,
- for each vertex v and label $a \in X^{\pm 1}$, a (partial) map `out`[v,a] giving the unique outgoing a-edge if it exists,
- and each edge stores its initial and terminal vertices and its label.

Steps.

1. **Free-reduce the inputs.** Replace each w_i by its freely reduced form. (This does not change the subgroup they generate.)

2. **Build the bouquet-of-words graph Γ_0.**

   (a) Initialize Γ with one vertex $*$.

   (b) For each word $w_i = a_1 a_2 \cdots a_k$ with $a_j \in X^{\pm 1}$: create a path of k directed edges labeled $a_1, \ldots, a_k$ starting at $*$ and ending at $*$. Concretely, create new intermediate vertices $v_1, \ldots, v_{k-1}$, and edges
   $$* \xrightarrow{a_1} v_1 \xrightarrow{a_2} \cdots \xrightarrow{a_k} *.$$

3. **Fold until deterministic.** Maintain a queue of *fold candidates*: pairs of distinct edges (e_1, e_2) with the same initial vertex and the same label. While the queue is nonempty:

   (a) Pop a candidate (e_1, e_2) with $\iota(e_1) = \iota(e_2) = v$ and $\mathrm{lab}(e_1) = \mathrm{lab}(e_2) = a$.

   (b) **Perform the fold:** identify e_1 and e_2 into one edge, and identify their terminal vertices $\tau(e_1)$ and $\tau(e_2)$. This is a quotient operation on the graph.

   (c) **Update adjacency:** whenever vertices are identified, merge their outgoing edge tables. If merging creates new duplicates (same label leaving the merged vertex), add them to the queue.

   (d) Continue until no duplicate-labeled outgoing edges remain anywhere.

   The resulting labeled graph is folded/deterministic; call it Γ_{fold}.

4. **Prune to the core.** Compute degrees in the underlying undirected graph. While there exists a vertex $v \neq *$ of degree 1:

   (a) remove v and its unique incident edge,

   (b) update degrees of its neighbor,

   (c) repeat until no such vertex remains.

   The remaining graph is the *core* Γ_H.

Correctness summary (what the algorithm guarantees).

- Every w_i labels a loop at $*$ in Γ_H (by construction).
- Folding does not change the subgroup recognized by loops at $*$ (it identifies indistinguishable states).
- Pruning removes only hanging trees that do not contribute to based loops, so the loop language is unchanged.

Boundary cases and implementation notes.

- **Already folded input.** If Γ_0 is already folded (no vertex has two outgoing edges with the same label), then the fold-candidate queue is empty and the folding loop does nothing. The algorithm proceeds directly to pruning.
- **Trivial subgroup.** If $m = 0$ (no generators) or all w_i freely reduce to the empty word, then Γ_0 is a single vertex $*$ with no edges. Folding and pruning do nothing and the output Stallings graph is the single base vertex; it recognizes exactly $H = \{1\}$.
- **Whole group.** If the input generators contain X itself (e.g. $H = \langle X \rangle = F(X)$), then the output core is the rose R_X (one vertex with one loop for each $x \in X$).
- **Keep the basepoint.** During pruning, never delete the basepoint $*$ even if it temporarily has degree 1; otherwise you would destroy the based-loop language.

Complexity (one standard bound; not necessarily optimal). [20, 23, 21] Let $L = \sum_{i=1}^{m} |w_i|$ after free reduction.

- **Building Γ_0.** Time $O(L)$ and space $O(L)$ vertices/edges.
- **Folding (naive).** If you implement folding by repeatedly scanning for conflicts, a safe bound is $O(L^2)$ time in the worst case.
- **Folding (typical efficient implementation).** Using union-find for vertex identifications and hash tables for `out`[v,a], one can achieve near-linear time, e.g. $O(L\alpha(L))$ or $O(L \log L)$, and $O(L)$ space.
- **Pruning to the core.** Time $O(|V| + |E|) = O(L)$ with a queue of degree-1 vertices; space $O(L)$.

Algorithm B (Inverse): Stallings graph → subgroup data (detailed)

[20, 14, 23]

Input. A finite based folded labeled graph $(\Gamma, *)$.

Outputs.

- The subgroup $H \leq F(X)$ recognized by loops at $*$.
- An explicit free basis of H (Schreier basis relative to a chosen tree).
- Membership/coset-state computation for words.

Steps.

1. **Recover H as a loop language.** Define:

$$H(\Gamma) = \{\text{lab}(p) \in F(X) \mid p \text{ is a reduced loop at } *\}.$$

This is a subgroup because concatenation of loops corresponds to multiplication in $F(X)$.

2. **Membership test (deterministic traversal).** Given a reduced word $w = a_1 \cdots a_k$:
   (a) set current vertex $v \leftarrow *$,
   (b) for $j = 1, \ldots, k$: if there is an outgoing edge from v labeled a_j, traverse it and update v; otherwise stop and declare $w \notin H$,
   (c) after reading all letters, declare $w \in H$ iff $v = *$.
3. **Compute a free basis via a spanning tree.**
   (a) Choose a maximal spanning tree T of the underlying undirected graph of Γ. (For algorithmic purposes, choose the BFS tree from $*$.)
   (b) For each vertex v, let $\sigma(v)$ be the label of the unique reduced path in T from $*$ to v.
   (c) For each oriented edge e not in T (choose one orientation per undirected edge), output the word
   $$g_e = \sigma(\iota(e))\,\mathrm{lab}(e)\,\sigma(\tau(e))^{-1}.$$
   (d) Then $\{g_e : e \notin T\}$ is a free basis of $H(\Gamma)$.
4. **Compute rank and (if finite index) the index.**
   (a) Rank: $|E| - |V| + 1$.
   (b) Check covering condition: for each vertex v and each $x \in X$, verify exactly one outgoing x-edge. If true, index is $|V|$.

Boundary cases and choices.

- **Trivial subgroup.** If Γ has one vertex and no edges, then $H = \{1\}$ and the Schreier basis is empty (rank 0).
- **Non-folded input.** If Γ is not folded, then reading a word from $*$ may have multiple paths; membership is not decidable by deterministic traversal. In practice you fold first (Algorithm A) or determinize the graph.
- **Choice of spanning tree.** Different choices of spanning tree T produce different Schreier bases; these bases are all free bases of the same subgroup and are related by Nielsen transformations. (The subgroup itself is independent of T.)

Complexity (rough). [20, 17, 4] Let Γ be finite with $|V|$ vertices and $|E|$ directed edges (or count undirected edges and multiply by 2).

- **Spanning tree.** Build a spanning tree by DFS/BFS in $O(|V| + |E|)$ time and $O(|V|)$ space.
- **Compute $\sigma(v)$.** If you store parent pointers in the tree, computing all $\sigma(v)$ as *paths* costs $O(|V|)$. If you materialize $\sigma(v)$ as explicit words, the total output size can be $O(|V|^2)$ in the worst case, because some tree paths may have length $\Theta(|V|)$.
- **Schreier basis.** There are $|E|-|V|+1$ non-tree edges, so basis size is $|E|-|V|+1$. Computing each g_e costs time proportional to the length of the corresponding tree paths (or $O(1)$ if you keep them as paths).
- **Membership.** Given a reduced word w of length n, deterministic traversal takes $O(n)$ time and $O(1)$ extra space.

Worked example 1: $H = \langle a^2, b \rangle \leq F(a, b)$ (no folds occur)

Forward construction (Algorithm A).

1. Inputs: $w_1 = a^2 = aa$, $w_2 = b$ (already freely reduced).
2. Build Γ_0:
   (a) Start with basepoint vertex $*$.
   (b) For aa: create one new vertex v and edges $* \xrightarrow{a} v$ and $v \xrightarrow{a} *$.
   (c) For b: add a loop edge $* \xrightarrow{b} *$.
3. Folding stage: check outgoing edges. At $*$ there is one a-edge and one b-edge; at v there is one a-edge. No duplicate labels, so no folds.
4. Pruning stage: degrees are $\deg(v) = 2$ and $\deg(*) \geq 2$, hence no removable leaves. So $\Gamma_H = \Gamma_0$.

Inverse reconstruction (Algorithm B).

1. Membership facts read off from traversal:
   - b labels a loop at $*$, so $b \in H$.
   - aa labels the loop $* \xrightarrow{a} v \xrightarrow{a} *$, so $a^2 \in H$.
   - a ends at $v \neq *$, so $a \notin H$.
2. Basis via a spanning tree:
   (a) Choose T containing the undirected edge between $*$ and v. Then $\sigma(*) = \varepsilon$ and $\sigma(v) = a$.
   (b) The non-tree edges are: the b-loop at $*$ and the edge $v \xrightarrow{a} *$.
   (c) Compute Schreier generators: $g_{b\text{-loop}} = b$ and $g_{v \to *} = a \cdot a = a^2$.

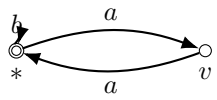

Figure 4: Stallings graph for $H = \langle a^2, b \rangle \leq F(a, b)$.

Worked example 2 (with an actual fold): $H = \langle a, ab \rangle \leq F(a, b)$

This example is useful because the fold step is *nontrivial* and illustrates the algorithm mechanically.

Forward construction (Algorithm A).

1. Inputs: $w_1 = a$, $w_2 = ab$ (already reduced).
2. Build Γ_0:
   (a) For a: add a loop edge $* \xrightarrow{a} *$.
   (b) For ab: create a new vertex v and edges $* \xrightarrow{a} v$ and $v \xrightarrow{b} *$.

3. Folding stage:

   (a) At the basepoint $*$ there are *two* outgoing a-edges: one is the a-loop with terminal $*$, the other is the first edge of the ab path with terminal v.

   (b) Fold these two a-edges together. This identifies their terminal vertices, so v is identified with $*$.

   (c) After this identification, the edge $v \xrightarrow{b} *$ becomes a b-loop at $*$.

   (d) Now at $*$ there is exactly one outgoing a-edge and one outgoing b-edge, so the graph is folded.

4. Pruning stage: there are no degree-1 vertices. Output Γ_H which is the rose with two loops labeled a and b.

Inverse reconstruction (Algorithm B).

1. In the output graph, both a and b are loops at the basepoint, hence $a, b \in H$.

2. Therefore H contains a free basis $\{a, b\}$ and so $H = F(a, b)$. (Indeed, algebraically $b = a^{-1}(ab) \in \langle a, ab \rangle$.)

Figure 5: After folding, the Stallings graph for $\langle a, ab \rangle$ is the rose with two loops: this subgroup is all of $F(a, b)$.

5 Enriched subgroup graph (Stallings + marking)

Definition

An *enriched subgroup graph* is a triple $(\Gamma_H, *, T)$ where:

- $(\Gamma_H, *)$ is the Stallings graph of $H \leq F(X)$,
- T is a chosen maximal spanning tree of Γ_H (undirected sense).

The enrichment provides *explicit* Schreier generators, a preferred set of tree-words $\sigma(v)$, and a deterministic procedure to decompose elements of H in a chosen free basis.[17, 20]

Existence/uniqueness (precise statement)

Proposition 5.1 (Enrichment exists; what is unique and what is not)**.** *Let $H \leq F(X)$ be finitely generated.*

1. *(**Existence**) The Stallings graph $(\Gamma_H, *)$ exists and is unique up to based label-preserving isomorphism (Theorem 4.1). Every finite connected graph admits a maximal spanning tree, hence an enrichment $(\Gamma_H, *, T)$ always exists.*
2. *(**Non-uniqueness**) The tree T is not canonical in general. Different choices of T typically yield different Schreier bases, though all such bases are Nielsen-equivalent.*
3. *(**Uniqueness relative to a chosen tree**) If $(\Gamma_H, *, T)$ and $(\Gamma_H, *, T')$ are two enrichments, then they are isomorphic as enriched objects iff there exists a based label-preserving automorphism Φ of Γ_H such that $\Phi(T) = T'$.*

Algorithm A (Forward): build and enrich (detailed)

[17, 20, 18]

Input. Generators $w_1, \ldots, w_m$ of $H \leq F(X)$.

Output. Enriched object $(\Gamma_H, *, T)$ plus the associated Schreier basis and tree-word table.

Steps.

1. Build $(\Gamma_H, *)$ using the Stallings folding+pruning algorithm (Algorithm A in the previous section).
2. Choose a maximal spanning tree T of Γ_H (for determinism, choose a BFS tree from $*$).
3. Compute the tree words: for each vertex v, compute $\sigma(v)$ as the label of the unique path in T from $*$ to v. (Algorithmically: store parent pointers and edge labels in the BFS tree.)
4. For each oriented edge $e \notin T$, compute the Schreier generator
$$g_e = \sigma(\iota(e))\,\mathrm{lab}(e)\,\sigma(\tau(e))^{-1}.$$
5. Output the set $\mathcal{B}_T = \{g_e : e \notin T\}$ as a free basis of H.

What changes if you choose a different spanning tree T? (non-uniqueness in a precise sense) The *underlying* Stallings graph $(\Gamma_H, *)$ is canonical up to based label-preserving isomorphism. However, the additional data of a spanning tree is a *choice*:

- The tree determines the table of *tree-words* $\sigma(v)$ (words in $X^{\pm 1}$ reaching each vertex).
- The Schreier generators are built from these $\sigma(v)$ via $g_e = \sigma(\iota(e))\mathrm{lab}(e)\sigma(\tau(e))^{-1}$. So changing T changes $\sigma(\cdot)$ and therefore changes the *explicit* basis $\mathcal{B}_T$.
- Different $\mathcal{B}_T$ are bases of the same free group H, and any two such bases are related by Nielsen moves (so they generate the same subgroup but give different coordinate systems).
- Likewise, Algorithm B produces different *factorizations* of the same $w \in H$ depending on T.

Boundary cases.

- If Γ_H has a single vertex and no edges (so $H = \{1\}$), then T is trivial and $\mathcal{B}_T = \varnothing$.
- If Γ_H is a tree (so $|E| = |V| - 1$), then $\mathcal{B}_T = \varnothing$ and necessarily $H = \{1\}$ for a core subgroup graph.

Complexity (rough). [17, 10, 4] Let Γ_H have $|V|$ vertices and $|E|$ directed edges.

- Choosing a BFS spanning tree and storing parent pointers costs $O(|V| + |E|)$ time and $O(|V|)$ space.
- Computing all $\sigma(v)$ as parent-pointer paths costs $O(|V|)$ (but writing them out as explicit words can cost $O(|V|^2)$ total output length in the worst case).
- Computing all Schreier generators g_e costs $O(|E|)$ operations *plus* the cost of materializing the tree-words used in the formula.

Algorithm B (Inverse): membership *and* explicit decomposition (detailed)

[17, 20, 18]

Input. An enriched graph $(\Gamma, *, T)$ and a word $w \in (X^{\pm 1})^*$.

Output. Either a certificate that $w \notin H$, or a decomposition of w as a word in the Schreier basis $\mathcal{B}_T$.

Steps.

1. Freely reduce w.
2. Deterministic traversal:
   (a) set $v_0 \leftarrow *$,
   (b) for each letter a_i of w, traverse the unique outgoing edge labeled a_i if it exists; if it does not exist, stop and output "$w \notin H$ (path undefined)". Let v_i be the vertex after reading the first i letters.

3. If $v_{|w|} \neq *$, output "$w \notin H$ (endpoint is $v_{|w|}$)".
4. If $v_{|w|} = *$, then $w \in H$. To compute a decomposition:
   (a) Initialize an empty word W in the alphabet $\mathcal{B}_T^{\pm 1}$.
   (b) For each traversed edge $e_i : v_{i-1} \xrightarrow{a_i} v_i$:
      - if the undirected edge underlying e_i lies in the tree T, record nothing (tree edges belong to the transversal part);
      - if the undirected edge is *not* in T, then:
         i. determine the chosen orientation e for that non-tree undirected edge (the one used to define g_e),
         ii. if $e_i = e$, append g_e to W; if $e_i = \overline{e}$ append g_e^{-1} to W.
   (c) Output W. Then the element represented by W in the free group on $\mathcal{B}_T$ equals the element represented by w in H.

Boundary cases and output format.

- **Empty word.** If $w = \varepsilon$, traversal stays at $*$ and the decomposition output is the empty product (the identity of H).
- **Traversal fails early.** If at step i there is no outgoing edge labeled a_i, then you can output the partial endpoint v_{i-1} as a "coset-state" and the failing letter a_i as an explicit certificate that $w \notin H$ in this deterministic automaton.
- **Endpoint not basepoint.** If traversal succeeds but ends at $v \neq *$, then $w \notin H$ and v records the state reached by w.

Complexity (rough). [17, 10, 4] Let $n = |w|$.

- Deterministic traversal costs $O(n)$ time (one outgoing-edge lookup per letter) and $O(1)$ auxiliary space.
- If you store a boolean "is-tree-edge" for each undirected edge, then deciding whether to output $g_e^{\pm 1}$ at each step is $O(1)$.
- The produced factorization has length at most n (you output at most one basis letter per traversed edge), so output size is $O(n)$.

Worked example: enrichment for $H = \langle a^2, b \rangle$

We reuse the Stallings graph in Worked example 1 of the previous section.

Forward enrichment (Algorithm A).

1. Choose the spanning tree T containing the undirected a-edge between $*$ and v.
2. Compute tree words: $\sigma(*) = \varepsilon$ and $\sigma(v) = a$.
3. Non-tree edges:
   - the b-loop at $*$ gives $g_b = b$,
   - the edge $v \xrightarrow{a} *$ gives $g_a = a \cdot a = a^2$.

   So $\mathcal{B}_T = \{b, a^2\}$.

Inverse use (Algorithm B): decide membership and decompose $w = b\,a^2\,b^{-1}$.

1. w is already freely reduced.
2. Traverse:
$$* \xrightarrow{b} * \xrightarrow{a} v \xrightarrow{a} * \xrightarrow{b^{-1}} *.$$
So endpoint is $*$ and $w \in H$.
3. Decompose using T:
   - The first edge is the non-tree b-loop: record b.
   - The next edge $* \xrightarrow{a} v$ is a tree edge: record nothing.
   - The next edge $v \xrightarrow{a} *$ is non-tree: record a^2.
   - The last edge is the inverse of the b-loop: record b^{-1}.

   Hence $w = b\,(a^2)\,b^{-1}$ in the Schreier basis.

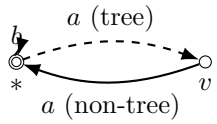

Figure 6: Enriched subgroup graph: dashed edge is the chosen spanning-tree edge. Non-tree edges give Schreier basis elements.

6 Schreier coset graph

Definition

Let G be generated by S and let $H \leq G$. The *Schreier coset graph* $\mathrm{Sch}(G, H, S)$ has:

- vertices: right cosets Hg,
- edges: $Hg \xrightarrow{s} Hgs$ for each $s \in S$.

The base vertex is H.[17, 18]

Existence/uniqueness (precise statement)

Theorem 6.1 (Schreier graphs: existence and uniqueness)**.** *Fix* (G, H, S).

1. *(**Existence**)* $\mathrm{Sch}(G, H, S)$ *exists. It is finite iff* $[G : H] < \infty$.
2. *(**Uniqueness**) The based labeled graph* $\mathrm{Sch}(G, H, S)$ *is uniquely determined up to based label-preserving isomorphism by the marked data* (G, H, S).
3. *(**Free-group covering classification**) If* $G = F(X)$, *then* $\mathrm{Sch}(F(X), H, X)$ *is the 1-skeleton of the covering of the rose* R_X *corresponding to* H. *Thus the Schreier graph is unique up to based labeled isomorphism because connected based coverings of* R_X *are classified by subgroups of* $\pi_1(R_X) \cong F(X)$.

Algorithm A (Forward): build Schreier graph (detailed)

[17, 22, 18, 11]

Input. A group G, subgroup H, generating set S.

Output. A directed labeled graph with vertex set $H\backslash G$.

Steps (conceptual).

1. Enumerate the set of cosets $\{Hg\}$. (If $[G : H]$ is finite, a coset enumeration algorithm such as Todd–Coxeter can produce the coset table.)
2. Create one vertex for each coset.
3. For each coset vertex Hg and each $s \in S$, compute the product coset Hgs and add the directed edge $Hg \xrightarrow{s} Hgs$.

Boundary cases.

- $H = G$**.** There is exactly one coset, so $\mathrm{Sch}(G, G, S)$ is a single vertex with a loop for each $s \in S$.
- $H = \{1\}$**.** Cosets are singletons, so $\mathrm{Sch}(G, \{1\}, S)$ is exactly the Cayley graph $\mathrm{Cay}(G, S)$.
- **Non-generating** S**.** If S does not generate G, the Schreier graph decomposes into multiple connected components corresponding to cosets of $\langle S \rangle$; we usually assume S generates.

Uniqueness vs. "choice of representatives" (what can change and what cannot). The *definition* uses vertices equal to the cosets Hg, so the Schreier graph is canonical up to based label-preserving isomorphism (Theorem 6.1). However, in computations and drawings one often labels a vertex by a *chosen representative word* $\widehat{g}$ for the coset Hg. If you change representatives (e.g. replace $\widehat{g}$ by $h\widehat{g}$ with $h \in H$), then:

- the **vertex names** change,
- the abstract labeled graph **does not change** (you only applied a relabeling of vertices),
- any two such "representative-labeled" drawings are related by a label-preserving graph isomorphism that sends one chosen transversal to the other.

Complexity (rough). [17, 22, 11] Let $m = [G : H]$ if finite.

- If a coset table is available (i.e. you can compute $Hg \cdot s$ for each coset and each $s \in S$), then constructing the Schreier graph takes $O(m|S|)$ time and stores $O(m|S|)$ directed edges.
- Producing the coset table itself (Todd–Coxeter) can be expensive in the worst case; we do not claim an optimal bound here.

Algorithm B (Inverse): membership and Schreier generators (detailed)

[17, 22, 18, 11]

Input. A Schreier graph $\mathrm{Sch}(G, H, S)$ with base vertex H.

Output. Membership test, coset reached by a word, and (optionally) Schreier generators for H.

Steps.

1. **Membership and coset output.** Given a word $w = s_1 \cdots s_k$ in $S^{\pm 1}$:
   (a) start at vertex $v_0 = H$,
   (b) follow the unique edge labeled s_i at each step (Schreier graphs are complete in the labels),
   (c) end at vertex $v_k = Hw$.

   Then $w \in H$ iff $v_k = H$.

2. **Schreier generators from a spanning tree.**
   (a) Choose a spanning tree T of the (undirected) Schreier graph.
   (b) For each vertex v, define $\sigma(v)$ as the label of the unique tree path in T from H to v.
   (c) For each directed edge $e : v \xrightarrow{s} w$ not in T, output

   $$g_e = \sigma(v)\, s\, \sigma(w)^{-1} \in H.$$

   These elements generate H (and in free-group settings yield a free basis).

Boundary cases and choices.

- **Membership is always well-defined.** Schreier graphs are complete in the labels: from each coset Hg and generator $s \in S$ there is an edge labeled s. So reading a word is always possible and ends at a well-defined coset.
- **Choice of spanning tree.** The Schreier generator formula depends on the choice of spanning tree T (equivalently, on the choice of a coset transversal $\{\sigma(v)\}$). Changing T changes the specific generating set produced, but not the subgroup H.

Complexity (rough). [17, 18, 4]

- **Membership / coset reached.** For a word w of length n, traversal takes $O(n)$ time and $O(1)$ extra space (besides the stored graph).
- **Schreier generators (finite index m).** Choosing a spanning tree takes $O(m|S|)$ time on the Schreier graph, and generating the list of non-tree edges is $O(m|S|)$.

Worked example: $G = \mathbb{Z} = \langle t \rangle$, $H = 2\mathbb{Z}$

Forward construction (Algorithm A).

1. The subgroup $H = 2\mathbb{Z}$ has index 2, so there are two right cosets.
2. Representatives: H (even integers) and Ht (odd integers).
3. Create vertices $v_0 = H$ and $v_1 = Ht$.
4. For generator t:
$$H \xrightarrow{t} Ht, \qquad Ht \xrightarrow{t} Ht^2 = H.$$

Inverse reconstruction (Algorithm B).

1. Reading a word t^k from base vertex H toggles between the two cosets.
2. Therefore $t^k \in H$ iff k is even, hence $H = \langle t^2 \rangle$.
3. If we choose the spanning tree T containing the edge $H \xrightarrow{t} Ht$, then $\sigma(H) = \varepsilon$, $\sigma(Ht) = t$, and the non-tree edge $Ht \xrightarrow{t} H$ gives Schreier generator $t \cdot t = t^2$.

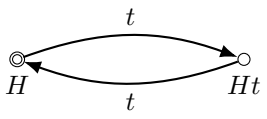

Figure 7: Schreier graph $\mathrm{Sch}(\mathbb{Z}, 2\mathbb{Z}, \{t\})$. Reverse edges carry t^{-1}.

7 Disk diagram

Definition

A *disk diagram* over a 2-complex X is a finite 2-complex D equipped with a combinatorial map $D \to X$ whose underlying topological space is a disk. Its boundary ∂D is a single combinatorial cycle; if edges are labeled, ∂D has a boundary word.[24, 15]

Existence/uniqueness (precise statement)

Theorem 7.1 (Disk diagrams exist exactly for null-homotopic loops)**.** *Let X be a combinatorial 2-complex and let γ be a combinatorial loop in X (or a boundary word in a presentation setting).*

1. *(**Existence**) γ is null-homotopic in X if and only if there exists a disk diagram $D \to X$ with boundary mapping to γ.*
2. *(**Minimal area exists**) If at least one disk diagram exists, then there exists a disk diagram of minimal area, where area means the number of 2-cells of D. (This follows from the well-ordering of $\mathbb{N}$.)*
3. *(**Non-uniqueness**) Disk diagrams filling the same loop are generally not unique up to isomorphism, even among minimal-area fillings.*

Algorithm A (Forward): build a disk diagram (two detailed input types)

[24, 15]

Input type A1 (topological). A null-homotopy $H : S^1 \times [0,1] \to X$ of a loop γ.

Steps (A1).

1. Identify the disk D^2 with $S^1 \times [0,1]/(S^1 \times \{1\}$ collapsed) so that H induces a map $D^2 \to X$.
2. Choose a triangulation (or polygonal cellulation) of D^2 that subdivides the boundary so the boundary map agrees with γ.
3. Apply cellular approximation to homotope the map to a combinatorial map sending vertices to vertices, edges to edge-paths, and faces to cellular 2-chains.
4. Record the resulting finite cell complex and map; this is the disk diagram.

Input type A2 (presentation). A presentation $\mathcal{P} = \langle X \mid R \rangle$ and an explicit proof that a boundary word w is trivial in $G(\mathcal{P})$.

Steps (A2).

1. Convert the proof into an equality in the free group:

$$w = \prod_{i=1}^{N} u_i \, r_i^{\varepsilon_i} \, u_i^{-1} \quad (r_i \in R,\ \varepsilon_i = \pm 1).$$

2. For each factor $r_i^{\varepsilon_i}$, create a polygonal 2-cell labeled by $r_i^{\varepsilon_i}$.

3. Attach a whisker path from a common boundary basepoint to the chosen start of the ith cell, labeled by u_i.

4. Glue boundary edges whenever adjacent letters cancel in the product; after gluing all cancellations, the remaining outer boundary reads w.

5. The result is a disk diagram; in the presentation case it is exactly a van Kampen diagram.

Boundary cases.

- **Trivial loop / empty word.** If γ is homotopic to a constant loop (or w freely reduces to the empty word), then the *empty diagram* (no 2-cells, boundary length 0) is a valid certificate.
- **Already a single relator.** If w is exactly a relator $r \in R^{\pm 1}$, then a one-face polygon labeled by r is a disk diagram.

Complexity (rough, for Input type A2). [24, 15, 21] Suppose you are given an explicit product-of-conjugates proof with total word length

$$L_{\text{proof}} := |w| + \sum_{i=1}^{N} \big(2|u_i| + |r_i|\big).$$

- Creating the N faces and whisker paths takes $O(L_{\text{proof}})$ time and space.
- Gluing cancellations can be done in $O(L_{\text{proof}} \log L_{\text{proof}})$ time using union-find / hashing on boundary arcs. A very safe (non-optimized) bound for a naive implementation is $O(L_{\text{proof}}^2)$ time.

Algorithm B (Inverse): disk diagram → explicit algebraic certificate (detailed)

[24, 15]

Input. A disk diagram $D \to K(\mathcal{P})$ with boundary word w.

Output. A concrete expression $w = \prod u_f r_f^{\pm 1} u_f^{-1}$ in the free group.

Steps.

1. Choose a basepoint $*$ on the boundary ∂D.
2. Choose a spanning tree T of the 1-skeleton $D^{(1)}$ that contains all boundary vertices (BFS tree works).
3. For each face f of D:
   (a) choose a point p_f on ∂f (a vertex on its boundary),
   (b) let $u_f = \text{lab}(\text{the unique path in } T \text{ from } * \text{ to } p_f)$,
   (c) read the cyclic word around ∂f starting at p_f in the direction consistent with the orientation; call it $r_f^{\varepsilon_f}$ (a relator or its inverse).

4. Traverse the diagram in the plane to order the faces (any order works if you interpret multiplication in the free group carefully). Then the boundary word satisfies

$$w = \prod_{f \in \mathrm{Faces}(D)} u_f \, r_f^{\varepsilon_f} \, u_f^{-1} \quad \text{in } F(X).$$

Boundary cases and correctness notes.

- If D has no faces, then w must be freely trivial; the output product is empty.
- The spanning tree choice affects the specific conjugators u_f, but not the fact that the product equals w in $F(X)$.

Complexity (rough). [15, 4] Let D have $|V|$ vertices, $|E|$ edges, $|F|$ faces, and total perimeter $P = \sum_f |\partial f|$.

- Building a spanning tree costs $O(|V| + |E|)$ time and $O(|V|)$ space.
- Reading all face boundary words costs $O(P)$ time.
- Therefore extracting a certificate costs $O(|V| + |E| + P)$ time.

Worked example: the commutator disk for $\langle a, b \mid aba^{-1}b^{-1} \rangle$

Forward construction (Algorithm A2).

1. Presentation: $\mathcal{P} = \langle a, b \mid r \rangle$ with $r = aba^{-1}b^{-1}$.
2. Boundary word: $w = r$.
3. Choose $N = 1$, $u_1 = \varepsilon$, $r_1 = r$, $\varepsilon_1 = +1$.
4. Create one square face whose boundary edges are labeled a, b, a^{-1}, b^{-1}.
5. The outer boundary reads w, so this is a disk diagram for w.

Inverse reconstruction (Algorithm B).

1. Choose basepoint $*$ on the boundary (a corner of the square).
2. Choose spanning tree T to be three edges of the square.
3. The diagram has one face f with label r and $u_f = \varepsilon$ (choose $p_f = *$).
4. Hence $w = u_f r u_f^{-1} = r$, so $w = 1$ in $G(\mathcal{P})$.

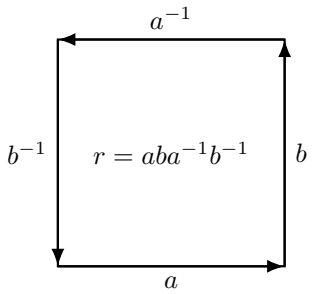

Figure 8: A disk diagram with one square face filling the commutator word.

8 Annular diagram

Definition

An *annular diagram* over a 2-complex X is a finite 2-complex $A \to X$ whose underlying space is an annulus. It has two boundary components:

$$\partial A = \partial_{\text{out}} A \sqcup \partial_{\text{in}} A,$$

each a cycle carrying a cyclic boundary word.

Existence/uniqueness (precise statement)

Theorem 8.1 (Annular diagrams detect free homotopy and conjugacy)**.** *1. (**Topological form**) Two loops $\gamma_0, \gamma_1 : S^1 \to X$ are freely homotopic in X if and only if there exists an annular diagram $A \to X$ whose boundary components map to γ_0 and γ_1.*

*2. (**Group form**) For a presentation $\mathcal{P} = \langle X \mid R \rangle$ and cyclic words u, v in $X^{\pm 1}$, u and v represent conjugate elements in $G(\mathcal{P})$ if and only if there exists an annular van Kampen diagram over $\mathcal{P}$ with boundary labels u and v (up to cyclic rotation and orientation conventions).*

*3. (**Non-uniqueness**) Annular diagrams are generally not unique for fixed boundary data.*

Algorithm A (Forward): conjugacy $u = gvg^{-1} \to$ annular diagram (detailed)

[15]

Input. A presentation $\mathcal{P} = \langle X \mid R \rangle$ and a conjugacy proof $u = gvg^{-1}$ in $G(\mathcal{P})$.

Output. An annular van Kampen diagram with boundary labels u and v.

Steps.

1. Form the word $W = gvg^{-1}u^{-1}$. Then $W = 1$ in $G(\mathcal{P})$.
2. Construct a van Kampen disk diagram D for W (Algorithm A in the van Kampen section).
3. On ∂D, locate the subpath labeled g and the subpath labeled g^{-1}. (These appear as consecutive segments in the boundary word $gvg^{-1}u^{-1}$.)
4. Glue the g-subpath to the g^{-1}-subpath by identifying corresponding edges with opposite orientations.
5. After gluing, the boundary splits into two cycles labeled by u and v (as cyclic words). The resulting complex is an annulus: an annular diagram.

Boundary cases and conventions.

- **Cyclic words.** The boundary labels of an annular diagram are *cyclic* words: choosing a different basepoint on a boundary component cyclically rotates the boundary word. Reversing the orientation inverts the cyclic word.
- **Trivial conjugator.** If $g = \varepsilon$ (so $u = v$ in the group), then $W = vu^{-1}$ and the gluing step is vacuous. In this case one can take a "degenerate" annulus (a cylinder) or just a disk diagram showing $u = v$.

Complexity (rough). [15] Assume you already have a disk van Kampen diagram for $W = gvg^{-1}u^{-1}$ with size $(|V|, |E|, |F|)$ and boundary length $|W|$.

- Locating the boundary subpaths labeled g and g^{-1} is $O(|W|)$.
- Gluing the two boundary subpaths identifies $|g|$ pairs of boundary edges, so it costs $O(|g|)$ union operations (and can be implemented in near-linear time in $|W|$).

The dominant cost is usually constructing the disk diagram for W (see van Kampen section).

Algorithm B (Inverse): annular diagram → explicit conjugator (detailed)

[15]

Input. An annular diagram A with boundary words u (outer) and v (inner).

Output. A word g such that $u = gvg^{-1}$ in the group.

Steps.

1. Choose basepoints $p \in \partial_{\text{out}}A$ and $q \in \partial_{\text{in}}A$.
2. Choose any path α in the 1-skeleton from p to q and set $g = \text{lab}(\alpha)$.
3. Cut the annulus along α. Topologically, cutting an annulus along an arc from outer to inner boundary yields a disk. Let D be the resulting disk diagram.
4. Read the boundary word of D (with consistent orientations): it is $g\, v\, g^{-1}\, u^{-1}$.
5. Because D is a disk diagram over $K(\mathcal{P})$, its boundary word is trivial in $G(\mathcal{P})$. Hence $u = gvg^{-1}$.

Non-uniqueness: in what sense, and what changes. Even after fixing the *cyclic* boundary words (outer u and inner v), annular diagrams are generally not unique up to isomorphism. Two important sources of non-uniqueness are:

- **Different fillings.** You can attach additional pairs of cancelling faces (dipoles) or use different cell structures, producing non-isomorphic annuli with the same boundary data.
- **Different choice of connecting path α.** In Algorithm B, the word $g = \text{lab}(\alpha)$ depends on α. If α' is another arc from outer to inner boundary, then $\alpha' \cdot \overline{\alpha}$ is a loop in the annulus, hence represents an element of $\pi_1(\text{annulus}) \cong \mathbb{Z}$. Algebraically, this means the resulting conjugators can differ by multiplication by a power of the boundary element (equivalently, by an element in the cyclic subgroup generated by v or u). So the conjugator is typically *not unique*, even when conjugacy holds.

Complexity (rough). [15] Let the annular diagram have $(|V|, |E|, |F|)$ and total perimeter P.

- Choosing a path α in the 1-skeleton and reading its label costs $O(|\alpha|) \le O(|E|)$.
- Cutting along α is a linear-time operation on a combinatorial representation (duplicate the edges/vertices along α), so $O(|\alpha|)$.
- Reading the resulting disk boundary word is $O(|u| + |v| + 2|\alpha|)$.

Worked example: $\langle a, b, t \mid tat^{-1} = b\rangle$

We construct an annular diagram showing that a and b are conjugate.

Forward construction (Algorithm A).

1. Presentation: $\mathcal{P} = \langle a, b, t \mid r\rangle$ with $r = tat^{-1}b^{-1}$.
2. We want $b = tat^{-1}$, i.e. $u = b$, $v = a$, $g = t$.
3. Form $W = gvg^{-1}u^{-1} = tat^{-1}b^{-1}$, which is exactly the relator r.
4. Build a one-face van Kampen disk diagram D with boundary label a cyclic shift of r.
5. Identify the boundary edge labeled t with the boundary edge labeled t^{-1} (opposite orientations). This gluing creates an annulus.
6. The remaining two boundary components are labeled (cyclically) by a and b.

Inverse reconstruction (Algorithm B).

1. In the annulus, choose α to be the glued seam corresponding to the letter t. Then $\mathrm{lab}(\alpha) = t$.
2. Cutting along α returns the one-face disk diagram with boundary word $tat^{-1}b^{-1}$.
3. Conclude $b = tat^{-1}$ in the group.

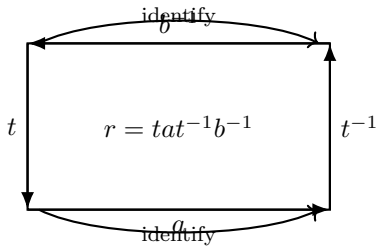

Figure 9: Cut-open annular diagram: gluing the t-side to the t^{-1}-side yields an annulus with boundary cycles labeled by a and b.

9 van Kampen diagram

Definition

Over $\mathcal{P} = \langle X \mid R \rangle$, a *van Kampen diagram* for a word $w \in (X^{\pm 1})^*$ is a disk diagram $D \to K(\mathcal{P})$ such that each 2-cell boundary label is a cyclic conjugate of some $r \in R^{\pm 1}$, and the outer boundary label is w.

Existence/uniqueness (precise statement)

Theorem 9.1 (van Kampen's lemma + normal closure interpretation)**.** *Let $\mathcal{P} = \langle X \mid R \rangle$ and let w be a word in $X^{\pm 1}$.*

1. *(**Existence equivalence**) $w = 1$ in $G(\mathcal{P})$ if and only if there exists a van Kampen disk diagram over $\mathcal{P}$ with boundary label w.*
2. *(**Normal closure equivalence**) Equivalently, w lies in the normal closure of R in $F(X)$:*

$$w \in \langle\langle R \rangle\rangle \quad \Longleftrightarrow \quad w = \prod_{i=1}^{N} u_i r_i^{\varepsilon_i} u_i^{-1} \text{ in } F(X).$$

3. *(**Minimal area exists**) If $w = 1$ in $G(\mathcal{P})$, then among all van Kampen diagrams for w there exists one with minimal area (fewest 2-cells).*
4. *(**Non-uniqueness**) Van Kampen diagrams for the same boundary word are generally not unique up to isomorphism, even among minimal-area diagrams.*

Non-uniqueness: in what sense? Even if you fix the *based* boundary cycle and the *exact linear* word w (not just its cyclic rotation), there can be many non-isomorphic van Kampen diagrams with boundary label w. Operations that preserve the boundary word but change the diagram include:

- inserting/removing cancellable dipoles (a pair of adjacent faces labeled by r and r^{-1} sharing an edge-path),
- different planar gluings realizing the same product-of-conjugates proof,
- different reduced/minimal-area fillings (minimal area itself is an invariant, but minimizers need not be unique).

So "unique up to isomorphism" does *not* hold for van Kampen diagrams; what is canonical is the *existence* equivalence and often the *minimal area value*.

Algorithm A (Forward): product of conjugates → van Kampen diagram (very detailed)

[24, 15]

Input. A representation in the free group

$$w = \prod_{i=1}^{N} u_i r_i^{\varepsilon_i} u_i^{-1}, \quad r_i \in R,\ \varepsilon_i = \pm 1.$$

Output. A planar simply connected labeled 2-complex with boundary label w.

Steps.

1. **Create faces.** For each i, draw a polygonal 2-cell f_i whose boundary cycle is subdivided and oriented so that reading around it gives a cyclic conjugate of $r_i^{\varepsilon_i}$.
2. **Choose attachment points.** On each face boundary ∂f_i, choose a distinguished vertex p_i (a "pin").
3. **Attach whiskers for conjugation.** Create a new basepoint vertex $*$ on the outside. For each i, attach a path P_i from $*$ to p_i whose label is u_i. (So traversing P_i then ∂f_i then P_i^{-1} reads $u_i r_i^{\varepsilon_i} u_i^{-1}$.)
4. **Form a boundary word and glue cancellations.** Consider the big loop based at $*$ that runs:
$$P_1\,\partial f_1\,P_1^{-1}\;P_2\,\partial f_2\,P_2^{-1}\;\cdots\;P_N\,\partial f_N\,P_N^{-1}.$$
Its label is exactly the product $\prod u_i r_i^{\varepsilon_i} u_i^{-1}$, which freely reduces to w.
Now perform *edge identifications* corresponding to free cancellations: whenever the concatenation of these loops produces adjacent letters aa^{-1}, identify the corresponding two boundary edges (with opposite orientations). Continue until all cancellations are realized.
5. **Extract the outer boundary.** After all identifications, the remaining unglued boundary cycle is the outer boundary ∂D and its label is w.
6. **Optional simplification: remove spurs/dipoles.** If the resulting complex contains immediate cancellable face pairs (dipoles) or boundary spurs, cancel them to obtain a reduced diagram.

Boundary cases.

- **Empty boundary word.** If w freely reduces to ε, then the empty diagram (no faces) certifies $w = 1$.
- **Single relator.** If w is (a cyclic conjugate of) $r^{\pm 1}$ for some $r \in R$, then one 2-cell suffices.

Complexity (rough). [24, 15] Assume the input is a product-of-conjugates proof with total length
$$L_{\text{proof}} := |w| + \sum_{i=1}^{N} \big(2|u_i| + |r_i|\big).$$

- Creating the faces and whiskers costs $O(L_{\text{proof}})$ time and space.
- Gluing all free cancellations can be implemented in near-linear time in the boundary size using union-find / hashing, but a safe bound for a naive implementation is $O(L_{\text{proof}}^2)$ time.
- The size of the resulting diagram is $O(L_{\text{proof}})$ cells/edges/vertices (before optional reductions).

Algorithm B (Inverse): van Kampen diagram → product of conjugates (very detailed)

[24, 15]

Input. A van Kampen diagram D over $\mathcal{P}$ with boundary label w.

Output. An explicit equality $w = \prod u_f r_f^{\varepsilon_f} u_f^{-1}$ in $F(X)$.

Steps.

1. Choose a basepoint $* \in \partial D$ on the outer boundary.
2. Choose a spanning tree T in $D^{(1)}$ containing all boundary vertices.
3. For each face $f \in \mathrm{Faces}(D)$:
   (a) pick a boundary vertex $p_f \in \partial f$,
   (b) let $u_f = \mathrm{lab}(\text{tree path in } T \text{ from } * \text{ to } p_f)$,
   (c) read the face boundary label starting at p_f (consistent orientation) to obtain a cyclic conjugate of some relator $r_f^{\varepsilon_f}$.
4. Then in the free group,
$$w = \prod_{f \in \mathrm{Faces}(D)} u_f \, r_f^{\varepsilon_f} \, u_f^{-1},$$
where the product order can be taken by traversing faces in any order consistent with a planar reduction (e.g. peel faces one-by-one from the boundary).

Boundary cases and choices.

- If D has no faces, then w must be freely trivial and the extracted product is empty.
- Different spanning trees produce different conjugator words u_f, but all yield valid certificates of the same equality in $F(X)$.

Complexity (rough). [15] Let D have $(|V|, |E|, |F|)$ and total perimeter P.

- Build a spanning tree: $O(|V| + |E|)$ time.
- Read all face boundary labels: $O(P)$ time.
- Total extraction time: $O(|V| + |E| + P)$, with space $O(|V| + |E|)$.

Worked example: one-face van Kampen diagram for $[a, b]$

This is the same as the commutator disk: $w = aba^{-1}b^{-1}$ is itself a relator.

Forward (Algorithm A).

1. Write $w = u_1 r_1 u_1^{-1}$ with $u_1 = \varepsilon$ and $r_1 = w$.
2. Create one square face labeled by w; no gluing is required.

Inverse (Algorithm B).

1. Choose basepoint on boundary; choose a spanning tree of the 1-skeleton.
2. There is one face f with $u_f = \varepsilon$ and $r_f = w$.
3. Output $w = \varepsilon \cdot w \cdot \varepsilon^{-1}$.

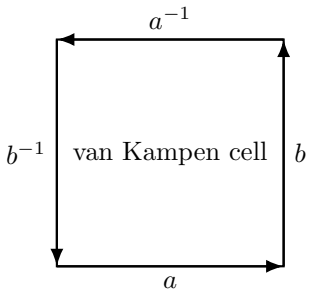

Figure 10: A one-face van Kampen diagram for $w = aba^{-1}b^{-1}$.

10 Dehn diagram

Definition

In a reduced van Kampen disk diagram D, a face f is a *Dehn cell* if ∂f shares a *contiguous* arc with the outer boundary ∂D of length *strictly more than half* the perimeter of f. A *Dehn diagram* is a van Kampen disk diagram viewed through this "peeling" lens.

Existence/uniqueness (concrete theorem)

Theorem 10.1 (Greendlinger-type Dehn cell existence under $C'(1/6)$)**.** *Let $\mathcal{P} = \langle X \mid R \rangle$ be a (classical) small-cancellation presentation satisfying $C'(1/6)$. Then every nontrivial reduced van Kampen disk diagram D over $\mathcal{P}$ contains a Dehn cell: there exists a face f such that ∂f shares with ∂D a contiguous boundary subpath of length $> \frac{1}{2}|\partial f|$. In particular, any nontrivial null-homotopic word admits a Dehn-reduction step that strictly shortens the boundary length.*

Remark 10.2. Other standard hypotheses also guarantee Dehn cells (e.g. $C'(1/4)$–$T(4)$ and $C'(1/3)$–$T(6)$). We state $C'(1/6)$ for concreteness.

Algorithm A (Forward): Dehn reductions → a peeling diagram (detailed)

[5, 8, 15]

Input. A word w and a sequence of Dehn reductions

$$w = w_0 \to w_1 \to \cdots \to w_k = \varepsilon$$

where each step replaces a subword longer than half a relator by the complementary shorter subword.

Output. A van Kampen disk diagram whose faces correspond to the reduction steps.

Steps.

1. Initialize a polygonal boundary cycle subdivided into $|w_0|$ directed edges labeled by w_0. This is the boundary of a "current partial diagram" with no faces.
2. For each reduction step $w_{i-1} \to w_i$:
   (a) Choose a relator $r \in R^{\pm 1}$ and a cyclic decomposition $r = uv$ such that u occurs as a contiguous subword of the current boundary word and $|u| > \frac{1}{2}|r|$.
   (b) Glue a new 2-cell labeled by r to the current diagram by identifying the u-arc of ∂r with the u-arc on the outer boundary.
   (c) After gluing, the complementary arc v becomes part of the new outer boundary.
   (d) Freely reduce the new outer boundary word; this reduced word is w_i.
3. After k steps, the boundary word is empty; the result is a disk diagram for w. By construction each added face is a Dehn cell at the moment it is added.

Boundary cases.

- If the reduction sequence has length $k = 0$ (so $w = \varepsilon$), then the output diagram is the empty diagram.
- If a reduction step uses $u = \partial r$ (gluing along the entire boundary), then that step adds a face and immediately removes that boundary component.

Complexity (rough). [5, 8, 15] Let $n_i = |w_i|$ and let $F = k$ be the number of Dehn reduction steps (so also the number of faces created).

- Building the initial boundary and gluing one face per step takes time proportional to the total boundary work performed. A safe bound is $O\left(\sum_{i=0}^{k-1} n_i\right)$ for a combinatorial implementation where each step edits the boundary cyclic word.
- In the worst case n_i decreases by only 1 each step, so $\sum n_i = O(n_0^2)$, giving a simple bound $O(n_0^2)$ time.

Algorithm B (Inverse): peel a Dehn diagram to produce reductions (detailed)

[5, 8, 15]

Input. A reduced van Kampen disk diagram D over a presentation satisfying the Dehn cell property.

Output. A sequence of Dehn reductions reducing the boundary word to empty.

Steps.

1. Let w_0 be the boundary word of D.
2. While D has at least one face:
   (a) Find a Dehn cell f (guaranteed by Theorem 10.1 under $C'(1/6)$).
   (b) Let u be the long boundary arc of ∂f that lies on the outer boundary ∂D; let v be the complementary arc of ∂f.
   (c) Remove f from the diagram and replace the boundary segment u by v.
   (d) Freely reduce the resulting boundary word; call it w_{i+1}.
3. Output the sequence $w_0 \to w_1 \to \cdots \to \varepsilon$. Each step strictly decreases length.

Boundary cases and implementation notes.

- If D has no faces, the boundary word is already freely trivial and the output reduction sequence is length 0.
- Finding a Dehn cell is the only nontrivial step. Under $C'(1/6)$ (or similar hypotheses), a Dehn cell exists in any nontrivial reduced diagram, so the loop terminates.

Complexity (rough). [15] Let D have F faces and total perimeter P.

- **Peeling a *given* diagram.** A naive implementation that scans all faces to find a Dehn cell at each peel step costs $O(P)$ per step, hence $O(PF)$ time in the worst case. With suitable bookkeeping of boundary adjacency, one can do better, but $O(PF)$ is a safe bound.
- **Dehn's word reduction algorithm (word-only view).** If you implement Dehn reduction by repeatedly scanning the current word of length $\leq n$ for a "long" subword of some relator, a simple bound is $O(n^2\|R\|)$ time (at most n reductions, each scan costs $O(n\|R\|)$).

Worked example: $\langle a \mid a^6 \rangle$ and $w = a^6$

This example is small and illustrates the peeling idea concretely.

Forward (Algorithm A).

1. Presentation has one relator $r = a^6$.
2. Start with boundary cycle labeled $w_0 = a^6$ (a 6-gon boundary).
3. Glue one 2-cell labeled a^6 along the entire boundary word $u = a^6$. Here $|u| = 6 > \frac{1}{2} \cdot 6 = 3$, so it is (trivially) a Dehn attachment.
4. After gluing, no boundary remains: $w_1 = \varepsilon$.

Inverse (Algorithm B).

1. The diagram has exactly one face and its entire boundary lies on ∂D, so it is a Dehn cell.
2. Peel it off: boundary becomes empty in one step.

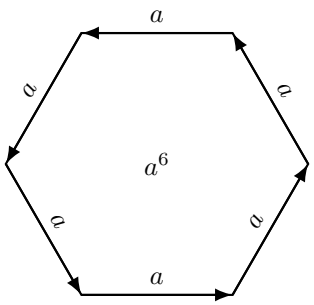

Figure 11: A Dehn diagram for a^6 with one hexagonal Dehn cell.

11 Spherical diagram

Definition

A *spherical diagram* over $\mathcal{P}$ is a finite labeled 2-complex $S \to K(\mathcal{P})$ whose underlying space is the sphere S^2 (no boundary). A spherical diagram is *reduced* if it contains no *dipole* (a cancellable adjacent pair of faces with inverse labels along a common edge-path).

Existence/uniqueness (precise statement)

Theorem 11.1 (Spherical diagrams, π_2, and identities among relators)**.** *Let $\mathcal{P} = \langle X \mid R \rangle$.*

1. *(**Existence** $\Leftrightarrow \pi_2$) There exists a nontrivial reduced spherical diagram over $\mathcal{P}$ if and only if $\pi_2(K(\mathcal{P})) \neq 0$.*
2. *(**Algebraic meaning**) A spherical diagram determines an* identity among relators*:*

$$\prod_{i=1}^{N} u_i \, r_i^{\varepsilon_i} \, u_i^{-1} = 1 \quad \text{in } F(X),$$

   and conversely such an identity can be realized by a spherical diagram (after gluing cancellations).
3. *(**Non-uniqueness**) Spherical diagrams are not unique; many non-isomorphic diagrams can represent the same element of π_2.*

Algorithm A (Forward): identity among relators → spherical diagram (detailed)

[6, 15]

Input. An identity in the free group:

$$\prod_{i=1}^{N} u_i \, r_i^{\varepsilon_i} \, u_i^{-1} = 1.$$

Output. A closed labeled surface diagram (a sphere in the simplest case).

Steps.

1. For each i, create a face f_i labeled by $r_i^{\varepsilon_i}$ and choose a pin vertex $p_i \in \partial f_i$.
2. Create a basepoint $*$ and attach a whisker path P_i from $*$ to p_i labeled by u_i.
3. Consider the boundary word of the concatenation of loops $P_i \, \partial f_i \, P_i^{-1}$. Because the product equals 1, this word freely reduces to the empty word.
4. Perform edge identifications for each cancellation until no boundary remains.
5. The resulting closed diagram is (typically) a sphere; in general the gluing might produce another closed surface. When it is S^2, you have a spherical diagram.

Boundary cases.

- The empty identity (no factors) gives the empty diagram (no faces). A *nontrivial* spherical diagram requires at least one face.
- The construction may yield a closed surface of genus > 0 if the identifications do not produce S^2. Spherical diagrams are the genus-0 case.

Complexity (rough). [6, 19, 15] Let the identity have total length

$$L_{\mathrm{id}} := \sum_{i=1}^{N} \big(2|u_i| + |r_i|\big).$$

- Building faces and whiskers costs $O(L_{\mathrm{id}})$ time/space.
- Performing all cancellation gluings costs at most quadratic time $O(L_{\mathrm{id}}^2)$ in a naive implementation; near-linear is possible with union-find, but $O(L_{\mathrm{id}}^2)$ is a safe bound.

Algorithm B (Inverse): spherical diagram → identity among relators (detailed)

[6, 15]

Input. A spherical diagram S over $\mathcal{P}$.

Output. An identity among relators.

Steps.

1. Choose a basepoint vertex $* \in S^{(1)}$ and a spanning tree $T \subseteq S^{(1)}$.
2. For each face f:
   (a) choose a boundary vertex $p_f \in \partial f$,
   (b) let $u_f = \mathrm{lab}(\text{tree path } * \to p_f)$,
   (c) read the relator word $r_f^{\varepsilon_f}$ around ∂f starting at p_f.
3. Multiply the conjugates $u_f r_f^{\varepsilon_f} u_f^{-1}$ over all faces. Because S has no boundary, the product equals 1 in $F(X)$.

Complexity (rough). [6] Let S have $(|V|, |E|, |F|)$ and total perimeter $P = \sum_f |\partial f|$.

- Building a spanning tree: $O(|V| + |E|)$.
- Reading all face boundaries: $O(P)$.
- Total extraction time: $O(|V| + |E| + P)$ with space $O(|V| + |E|)$.

Worked example: two-face dipole sphere ($r \cdot r^{-1} = 1$)

Forward (Algorithm A).

1. Start with the identity $r \cdot r^{-1} = 1$.
2. Create two faces: f_1 labeled r and f_2 labeled r^{-1}.
3. Glue their entire boundaries edge-by-edge (matching labels with opposite orientations).
4. The result is a sphere with two faces: a dipole.

Inverse (Algorithm B).

1. Choose basepoint and spanning tree.
2. Each face contributes a conjugate of r or r^{-1}.
3. The extracted identity is (up to trivial conjugations) $r \cdot r^{-1} = 1$.

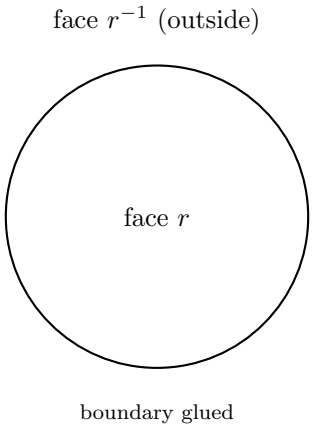

Figure 12: Planar depiction of a spherical diagram: interior region labeled r and the exterior region labeled r^{-1}.

12 Howie diagram (relative diagrams)

Definition (detailed)

Let G be a base group and let

$$\mathcal{P}_{\mathrm{rel}} = \langle G, X \mid \mathcal{R} \rangle$$

be a *relative presentation*, where relators lie in the free product $G * F(X)$. Write a relator in alternating form:

$$r = g_0 x_1 g_1 x_2 \cdots x_n g_n, \quad g_i \in G,\ x_i \in X^{\pm 1}.$$

A *Howie diagram* (one standard convention) is a disk or sphere diagram where:

- oriented edges are labeled by letters of $X^{\pm 1}$,
- *corners* (angles between consecutive edges along a face) are labeled by elements of G,
- reading around each face (alternating corner labels and edge labels) yields a cyclic conjugate of some $r \in \mathcal{R}^{\pm 1}$,
- around each vertex, the product of corner labels (in cyclic order) equals 1 in G.

Existence/uniqueness (precise statement)

Theorem 12.1 (Relative van Kampen / Howie lemma)**.** *Let $\mathcal{P}_{\mathrm{rel}} = \langle G, X \mid \mathcal{R} \rangle$ and let W be an alternating boundary word in $G * F(X)$. Then*

$$W = 1 \text{ in } \langle G, X \mid \mathcal{R} \rangle \quad \Longleftrightarrow \quad \text{there exists a Howie disk diagram over } \mathcal{P}_{\mathrm{rel}} \text{ with boundary } W.$$

Howie diagrams are generally not unique for fixed boundary data.

Non-uniqueness (what can vary). Even with the boundary word W fixed as an element of the free product $G * F(X)$ (not just up to cyclic rotation), Howie diagrams are generally not unique: different planar gluings, insertion/removal of dipoles, and different choices of spanning tree/basepoint produce non-isomorphic diagrams encoding the same equality.

Algorithm A (Forward): relative equality → Howie diagram (detailed)

Input. An explicit identity in the free product $G * F(X)$ of the form

$$W = \prod_{i=1}^{N} U_i \, R_i^{\varepsilon_i} \, U_i^{-1},$$

where each $R_i \in \mathcal{R}$ is a relator word and $U_i \in G * F(X)$.

Output. A Howie disk diagram with boundary W.

Steps.

1. Write each relator R_i in alternating form $g_0 x_1 g_1 \cdots x_n g_n$.
2. For each relator occurrence, create a face:
   - put the x_j on directed edges in cyclic order,
   - put the g_j on corners between edges.
3. Attach "whiskers" (paths in the edge-labeled 1-skeleton) encoding the conjugators U_i.
4. Glue boundary edges labeled by $X^{\pm 1}$ according to cancellations in the product.
5. The vertex corner-label condition (product equals 1 in G) is automatically satisfied if the algebraic identity holds; combinatorially, it ensures the diagram consistently records coefficients in G.

Boundary cases.

- If $W = \varepsilon$ and $N = 0$, the output is the empty diagram.
- If there is a single relator factor (and $U_1 = \varepsilon$), the diagram has one face.

Complexity (rough). [12, 13] Let

$$L_{\mathrm{rel}} := \sum_{i=1}^{N} \big(2|U_i| + |R_i|\big),$$

where lengths are measured in the alternating normal form in $G * F(X)$ (counting X-letters and recording G-coefficients).

- Creating faces and whiskers costs $O(L_{\mathrm{rel}})$ time/space.
- Performing all cancellation gluings among X-edges costs at most $O(L_{\mathrm{rel}}^2)$ time in a naive implementation.

Algorithm B (Inverse): Howie diagram → relative equality (detailed)

Input. A Howie disk diagram D with boundary word W.

Output. A derivation showing $W = 1$ in the relative group.

Steps.

1. Choose a basepoint on the boundary and a spanning tree T in the edge-labeled 1-skeleton.
2. For each face f:
   (a) choose a boundary corner/vertex on ∂f,
   (b) read the alternating face word $R_f^{\varepsilon_f}$ (corners in G, edges in $X^{\pm 1}$),
   (c) compute the conjugator U_f as the word in $G * F(X)$ obtained by reading the tree path from the basepoint to that chosen place.
3. Multiply the conjugates $\prod_f U_f R_f^{\varepsilon_f} U_f^{-1}$; it equals W (disk case) or 1 (sphere case).

Complexity (rough). [12, 13] Let D have $(|V|, |E|, |F|)$ and let P be the total number of X-labeled edge occurrences around all faces (the "X-perimeter").

- Build a spanning tree: $O(|V| + |E|)$.
- Read all face boundary data: $O(P)$ to read X-edges plus the cost of multiplying/recording corner labels in G (treated here as $O(1)$ per corner if group operations in G are $O(1)$).
- Total extraction time: $O(|V| + |E| + P)$.

Worked example: $\langle G, t \mid t^{-1}atb^{-1} \rangle$

Fix $a, b \in G$ and consider the relative presentation with one relator $t^{-1}atb^{-1}$.

Forward (Algorithm A).

1. Here $X = \{t\}$ and $\mathcal{R} = \{t^{-1}atb^{-1}\}$.
2. Create one face. Its boundary alternates: edge t^{-1}, corner a, edge t, corner b^{-1}.
3. The boundary word of the diagram is $t^{-1}atb^{-1}$, hence it is trivial in the quotient.

Inverse (Algorithm B).

1. Read the unique face boundary word $t^{-1}atb^{-1}$.
2. Conclude $t^{-1}atb^{-1} = 1$ in the relative group, hence $t^{-1}at = b$.

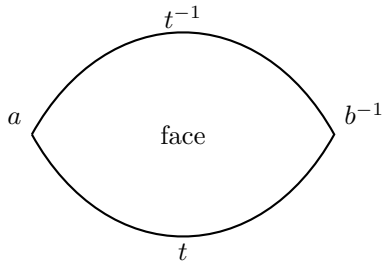

Figure 13: A one-face Howie diagram encoding the relative relator $t^{-1}atb^{-1}$. Edges are labeled by $t^{\pm 1}$ and corners by elements of G.

13 Aspherical diagram (asphericity via absence of reduced spheres)

Definition

A 2-complex X is *aspherical* if $\pi_2(X) = 0$. For a presentation $\mathcal{P}$, we say $\mathcal{P}$ is aspherical if the presentation complex $K(\mathcal{P})$ is aspherical.

Existence/uniqueness (property theorem, detailed)

Theorem 13.1 (Asphericity and spherical diagrams)**.** *Let $\mathcal{P} = \langle X \mid R \rangle$. The following are equivalent:*

1. *$K(\mathcal{P})$ is aspherical: $\pi_2(K(\mathcal{P})) = 0$.*
2. *There exists* no *nontrivial reduced spherical diagram over $\mathcal{P}$.*
3. *Every spherical diagram over $\mathcal{P}$ can be reduced to the empty diagram by cancelling dipoles.*

Moreover, a stronger sufficient condition is diagrammatic reducibility (DR)*: every spherical diagram contains a dipole. DR implies asphericity.*

Algorithmic remark (complexity for a *given* spherical diagram). If you are handed a specific spherical diagram S, then you can attempt to reduce it by cancelling dipoles:

- A single dipole cancellation can be found by scanning adjacent face pairs along shared edges; a naive scan costs $O(P)$ where P is total perimeter.
- Each cancellation strictly decreases the number of faces, so at most $|F|$ cancellations occur. Thus a safe bound for reducing a *given* diagram is $O(P|F|)$ time.

Warning. Theorem 13.1 is a characterization, but deciding whether an arbitrary finite presentation is aspherical is a subtle global problem; in general one should not expect a simple efficient algorithm without additional hypotheses (small cancellation, CAT(0) cube complex structure, etc.).

Example 1: free group wedge of circles is aspherical

Forward construction (space viewpoint).

1. The presentation complex $K(\langle a, b \mid \rangle)$ has one vertex and two 1-cells, with no 2-cells. So K is a graph (1-dimensional CW complex).
2. Any 2-sphere map into a 1-dimensional complex is null-homotopic, hence $\pi_2(K) = 0$.

Inverse diagram viewpoint.

1. A spherical diagram would require faces labeled by relators.
2. There are no relators and no 2-cells, so no spherical diagrams exist.

Figure 14: A wedge of two circles: the presentation complex for $\langle a, b \mid \rangle \cong F_2$.

Example 2: the torus presentation $\langle a, b \mid [a, b] \rangle$ is aspherical

Forward (universal cover viewpoint).

1. The Cayley graph of $\langle a, b \mid [a, b] \rangle$ is the square grid.
2. Attach a square 2-cell to every unit square cycle in the grid (each labeled by the commutator).
3. The resulting simply connected 2-complex is the square tiling of the plane $\mathbb{R}^2$, which is contractible.
4. Therefore the universal cover of $K(\mathcal{P})$ is contractible, implying $\pi_2(K(\mathcal{P})) = 0$.

Inverse (diagram obstruction viewpoint).

1. If a reduced spherical diagram existed, it would represent a nontrivial element of $\pi_2(K(\mathcal{P}))$.
2. Lifting to the contractible universal cover would make it null-homotopic, contradiction.
3. Hence no reduced spherical diagrams exist, consistent with Theorem 13.1.

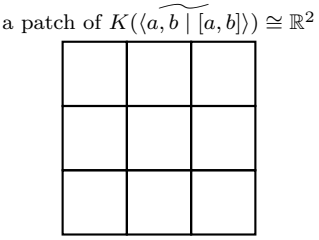

Figure 15: A finite patch of the square tiling universal cover of the torus presentation complex.

14 Triangle diagram

Definition

A *triangle diagram* is a van Kampen diagram in which every face is a triangle (3-gon). This is natural for *triangular presentations* where every relator has length 3.

Existence/uniqueness (detailed)

Proposition 14.1 (Triangle diagrams: existence and relation to general diagrams)**.** *1. If $\mathcal{P} = \langle X \mid R \rangle$ is triangular (every $r \in R$ has length 3), then every van Kampen diagram over $\mathcal{P}$ is automatically a triangle diagram.*

2. More generally, any polygonal disk diagram can be subdivided into triangles (e.g. by barycentric subdivision of the 2-complex D). This changes the cell structure of the diagram but not the boundary word it fills.

Triangle diagrams are generally not unique.

Algorithm A (Forward): build a triangle diagram (detailed)

[10]

Input. Either:

- a triangular presentation and a proof of triviality $w = 1$ (as a product of conjugates), or
- a general polygonal van Kampen diagram to be triangulated.

Output. A triangle diagram.

Steps (triangular presentation).

1. Express $w = \prod u_i r_i^{\varepsilon_i} u_i^{-1}$ where each r_i has length 3.
2. Apply van Kampen Algorithm A: each face $r_i^{\varepsilon_i}$ is drawn as a triangle.
3. Glue cancellations; all faces remain triangles.

Steps (triangulating a polygonal diagram).

1. For each n-gon face, insert a new vertex at its center and connect it to all boundary vertices (barycentric subdivision).
2. This subdivides the face into n triangles.
3. Repeat for all faces; keep all existing edge labels on original edges. New edges are "auxiliary" edges in the subdivided diagram (in topology they map into X by the cellular map).

Boundary cases.

- If the input diagram already has only triangular faces, the triangulation procedure does nothing.
- If $n = 3$, you may choose to leave the triangular face as-is (do nothing). If you apply the coning (star) subdivision anyway, it splits into 3 triangles. If you instead apply true barycentric subdivision, it splits into 6 triangles.

Complexity (rough). [10]

- **From a triangular presentation + proof.** Same as van Kampen Algorithm A: if the proof has total length L_{proof}, then a safe bound is $O(L_{\text{proof}}^2)$ time for naive gluing and $O(L_{\text{proof}})$ space.
- **Triangulating an existing polygonal diagram.** Let $P = \sum_f |\partial f|$ be the total perimeter. Barycentric subdivision adds exactly $|\partial f|$ triangles in each face f, so the new number of faces is $O(P)$. The subdivision can be performed in $O(P)$ time and produces $O(P)$ new edges/vertices.

Algorithm B (Inverse): triangle diagram → relator product (detailed)

[10]

Input. A triangle diagram D over a presentation $\mathcal{P} = \langle X \mid R \rangle$ (so each face boundary is a cyclic conjugate of some $r \in R^{\pm 1}$ and has length 3).

Output. An explicit equality in the free group

$$w = \prod_{f \in \mathrm{Faces}(D)} u_f \, r_f^{\varepsilon_f} \, u_f^{-1},$$

where w is the outer boundary word of D.

Steps.

1. Choose a basepoint $* \in \partial D$ on the outer boundary.
2. Choose a spanning tree T in the 1-skeleton $D^{(1)}$ containing all boundary vertices.
3. For each triangular face f:
   (a) choose a vertex p_f on ∂f,
   (b) define $u_f = \mathrm{lab}(\text{the unique path in } T \text{ from } * \text{ to } p_f)$,
   (c) read the (length 3) cyclic word around ∂f starting at p_f to obtain a cyclic conjugate of a relator $r_f^{\varepsilon_f}$.
4. Multiply the conjugates in any order obtained by peeling faces from the boundary (so that the product corresponds to the cancellation of internal edges). The resulting product equals the boundary word w in $F(X)$.

Complexity (rough). [10] If D has $(|V|, |E|, |F|)$ and total perimeter P, then extracting a relator product by choosing a spanning tree and reading each triangular face costs $O(|V| + |E| + P)$ time and $O(|V| + |E|)$ space, as in the van Kampen case.

Worked example: $\langle a, b, c \mid abc \rangle$

Forward (triangular presentation).

1. Relator $r = abc$ has length 3.
2. Boundary word $w = abc$.
3. Build one triangular face with boundary labels a, b, c.

Inverse .

1. Choose basepoint on boundary.
2. There is one face and the conjugator is trivial; hence $w = r$.

15 Polygonal complex

Definition

A *polygonal complex* is a 2-dimensional CW-complex whose 2-cells are polygons attached along edge cycles.

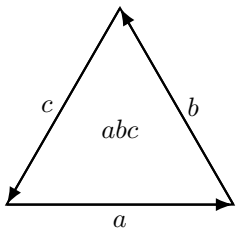

Figure 16: A one-face triangle diagram for the relator abc.

Existence/uniqueness (detailed theorem)

Theorem 15.1 (Presentation complexes and reconstruction)**.** *1. (**Existence**) For every presentation $\mathcal{P} = \langle X \mid R \rangle$ there exists a polygonal 2-complex $K(\mathcal{P})$ (the presentation complex) with $\pi_1(K(\mathcal{P})) \cong G(\mathcal{P})$.*

*2. (**Uniqueness with fixed combinatorics**) If the cyclic representative of each relator and the orientations/labels of the generator loops are fixed, then $K(\mathcal{P})$ is unique up to a label-preserving cellular isomorphism.*

*3. (**Reconstruction**) Conversely, for any connected polygonal complex X there is a standard algorithm to extract a presentation for $\pi_1(X)$: choose a maximal tree in the 1-skeleton, make generators from non-tree edges, and relators from 2-cell attaching maps. Different tree choices yield Tietze-equivalent presentations.*

Algorithm A (Forward): presentation $\to K(\mathcal{P})$ (detailed)

[10, 1]

Input. $\mathcal{P} = \langle X \mid R \rangle$.

Output. A labeled polygonal complex $K(\mathcal{P})$.

Steps.

1. Create one vertex v_0.
2. For each generator $x \in X$, attach an oriented loop edge at v_0 labeled x.
3. For each relator $r = a_1 \cdots a_n$ with $a_i \in X^{\pm 1}$:
   (a) create an n-gon 2-cell f_r whose boundary is subdivided into n directed edges,
   (b) label these boundary edges in order by $a_1, \ldots, a_n$,
   (c) glue the boundary to the 1-skeleton by mapping each labeled edge to the corresponding generator loop (with orientation determined by a_i).
4. The resulting CW-complex is $K(\mathcal{P})$.

Boundary cases.

- **No relators.** If $R = \varnothing$, then $K(\mathcal{P})$ is a wedge of $|X|$ circles (a 1-dimensional CW-complex).
- **Trivial group.** If $X = \varnothing$ and $R = \varnothing$, then $K(\mathcal{P})$ is a single point and π_1 is trivial.

Complexity (rough). [10, 1] Let $\|R\| = \sum_{r \in R} |r|$.

- Building the 1-skeleton takes $O(|X|)$ time and space.
- Creating and attaching all 2-cells takes $O(\|R\|)$ time and produces $O(\|R\|)$ boundary edge occurrences.
- Total size of $K(\mathcal{P})$ is $O(|X| + \|R\|)$ cells, so the forward construction is linear in the input size.

Algorithm B (Inverse): polygonal complex → a presentation for π_1 (detailed)

[10, 1]

Input. A connected polygonal complex X.

Output. A presentation for $\pi_1(X)$.

Steps.

1. Choose a maximal spanning tree T in the 1-skeleton $X^{(1)}$.
2. For each oriented edge e of $X^{(1)}$ not in T, introduce a generator x_e (and identify $x_{\overline{e}} = x_e^{-1}$).
3. For each 2-cell f:
   (a) traverse its attaching cycle once around, recording a word in the generators: when you traverse a non-tree edge e, write x_e (or x_e^{-1} if traversed backwards); when you traverse a tree edge, write nothing (tree edges become trivial in the quotient).
   (b) The resulting word is a relator r_f.
4. Output the presentation

$$\pi_1(X) \cong \langle x_e \ (e \notin T) \mid r_f \ (f \in X^{(2)}) \rangle.$$

Complexity (rough). [10, 1] Let X have $|V|$ vertices and $|E|$ edges in its 1-skeleton, and let $P = \sum_f |\partial f|$ be total perimeter of 2-cells.

- Spanning tree computation: $O(|V| + |E|)$ time.
- Reading all 2-cell attaching cycles: $O(P)$ time.
- Total extraction time: $O(|V|+|E|+P)$, with output size proportional to the number of non-tree edges and the total relator length you read.

Worked example: fundamental polygon for the torus

Forward (Algorithm A).

1. Presentation: $\langle a, b \mid aba^{-1}b^{-1} \rangle$.
2. 1-skeleton: one vertex, two loop edges labeled a and b.
3. Attach one square 2-cell whose boundary is labeled a, b, a^{-1}, b^{-1}.
4. The resulting complex is a 2-torus (a polygonal complex with one square face).

Inverse (Algorithm B).

1. Choose maximal tree T to be just the vertex (no edges). Then both loop edges are non-tree edges: generators are a and b.
2. The square face boundary reads $aba^{-1}b^{-1}$, giving the relator.
3. Hence $\pi_1 \cong \langle a, b \mid aba^{-1}b^{-1} \rangle$.

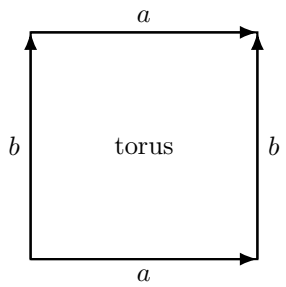

Figure 17: A square fundamental polygon for the torus (a polygonal complex with one square 2-cell).

16 RAAG diagram (Salvetti / cube complex)

Definition

Given a finite simplicial graph Γ, the right-angled Artin group (RAAG) is

$$A_\Gamma = \langle V(\Gamma) \mid [v, w] = 1 \text{ whenever } \{v, w\} \in E(\Gamma) \rangle.$$

The *Salvetti complex* S_Γ is the canonical cube complex built from Γ: one loop per generator, one square per commuting pair, and higher-dimensional cubes for cliques.

Existence/uniqueness (detailed theorem)

Theorem 16.1 (Salvetti complex: existence, uniqueness, and reconstruction)**.** *Let Γ be a finite simplicial graph and A_Γ its RAAG.*

1. *(**Existence**) There exists a cube complex S_Γ such that $\pi_1(S_\Gamma) \cong A_\Gamma$.*

2. *(**Asphericity**) S_Γ is a $K(A_\Gamma, 1)$: its universal cover $\widetilde{S_\Gamma}$ is contractible (indeed CAT(0)).*

3. *(**Uniqueness**) The cubical isomorphism class of S_Γ is uniquely determined by Γ.*

4. *(**Reconstruction of** Γ) The graph Γ can be recovered from S_Γ: its vertices correspond to 1-cells at the unique vertex of S_Γ, and two vertices are adjacent in Γ iff the corresponding 1-cells span a square 2-cell in S_Γ (equivalently: they commute in π_1).*

Algorithm A (Forward): $\Gamma \to S_\Gamma$ (detailed)

[7, 16, 3, 1, 9]

Input. A finite simplicial graph Γ.

Output. The cube complex S_Γ.

Steps.

1. Create one vertex v_0.
2. For each vertex $x \in V(\Gamma)$, attach a loop edge at v_0 labeled x.
3. For each edge $\{x, y\} \in E(\Gamma)$, attach a square 2-cell whose boundary is labeled $xyx^{-1}y^{-1}$. This enforces the commutation relation $[x, y] = 1$.
4. For each clique $C = \{x_1, \ldots, x_k\}$ (complete subgraph): attach a k-cube whose 1-skeleton uses the loop edges labeled x_i and whose 2-faces are exactly the commuting squares from the edges in C. (Equivalently: attach a k-torus cell structure for the abelian subgroup $\langle C \rangle \cong \mathbb{Z}^k$.)

Boundary cases.

- **Empty graph.** If $V(\Gamma) = \varnothing$, then A_Γ is trivial and S_Γ is a single vertex (a point).
- **Complete graph.** If Γ is complete on n vertices, then $A_\Gamma \cong \mathbb{Z}^n$ and S_Γ is an n-torus cellulation (one k-cube for each k-subset).

Complexity (rough). [16, 3, 1, 9] Let $n = |V(\Gamma)|$ and $m = |E(\Gamma)|$.

- Building the **1-skeleton** takes $O(n)$ time and space.
- Building the **2-skeleton** (adding one square per edge) takes $O(m)$ time and adds m squares.
- Building the **full Salvetti complex** requires adding a k-cube for every clique of size k. The number of cliques can be exponential in n in the worst case, so constructing the entire complex is exponential in general. (For many applications one works only up to some fixed dimension, e.g. the 2-skeleton.)

Algorithm B (Inverse): $S_\Gamma \to \Gamma$ and the RAAG presentation (detailed)

[7, 16, 3, 1, 9]

Input. A Salvetti-type cube complex S with one vertex.

Output. The defining graph Γ and the RAAG presentation.

Steps.

1. List the oriented 1-cells (loops) at the unique vertex; call their labels V. These are the generators.
2. For each unordered pair $\{x, y\} \subseteq V$, check if there is a square 2-cell whose boundary word is $xyx^{-1}y^{-1}$. If yes, put an edge between x and y.
3. The resulting graph is Γ and the recovered group presentation is

$$\langle V \mid [x, y] = 1 \text{ for each edge } \{x, y\} \in E(\Gamma)\rangle.$$

Complexity (rough). [16, 3] Suppose S has one vertex, n loop 1-cells, and q square 2-cells.

- Listing the generators (1-cells) costs $O(n)$.
- Checking commutations by scanning the list of squares costs $O(q)$ if the squares are stored with their edge labels. A naive pairwise check over all unordered pairs of generators is $O(n^2)$, but in practice q is the relevant parameter.
- Output size: n generators and q commutator relations.

Example 1: no edge $\Rightarrow A_\Gamma \cong F_2$

Forward (Algorithm A).

1. Γ has vertices $\{a, b\}$ and no edge.
2. Build S_Γ: one vertex and two loop edges labeled a and b.
3. There are no edges in Γ, hence attach no squares and no higher cubes.
4. Therefore $\pi_1(S_\Gamma) \cong \langle a, b \mid \rangle \cong F_2$.

Inverse (Algorithm B).

1. From S_Γ, generators are the two 1-cells a and b.
2. There is no square 2-cell, so no commuting relations.
3. Hence recovered Γ has no edge and recovered group is F_2.

Figure 18: Salvetti complex for Γ with two isolated vertices: a wedge of two circles, $\pi_1 \cong F_2$.

Example 2: one edge $\Rightarrow A_\Gamma \cong \mathbb{Z}^2$

Forward (Algorithm A).

1. Γ has vertices $\{a, b\}$ with a single edge $\{a, b\}$.
2. Build 1-skeleton: one vertex with loops labeled a and b.
3. Because $\{a, b\}$ is an edge, attach one square with boundary word $aba^{-1}b^{-1}$.
4. No higher cubes exist (no 3-cliques), so the result is a 2-torus cell structure. Thus $\pi_1 \cong \langle a, b \mid [a, b]\rangle \cong \mathbb{Z}^2$.

Inverse (Algorithm B).

1. Generators are the two 1-cells a and b.
2. There is a square with commutator boundary, hence a and b commute and $\{a, b\}$ is an edge in Γ.
3. Therefore recovered Γ is an edge graph and the recovered group is $\mathbb{Z}^2$.

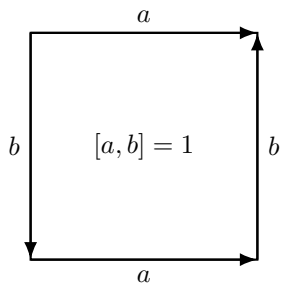

Figure 19: Salvetti complex for an edge graph on $\{a, b\}$: a square 2-cell yields $\mathbb{Z}^2$.

17 Automatic group diagram (automatic structures)

Definition

Fix a finitely generated group $G = \langle A \rangle$ with a finite *symmetric* generating set $A = A^{-1}$. An *automatic structure* for the marked group (G, A) consists of:

- a regular language $L \subseteq A^*$ (the *combing language*) such that the evaluation map $\pi : A^* \to G$ is surjective on L (every $g \in G$ has at least one representative in L);
- for each $a \in A \cup \{1\}$, a finite automaton recognizing the *multiplier relation*

$$R_a \;=\; \{(u, v) \in L \times L :\; \pi(u)\, a = \pi(v) \text{ in } G\},$$

  viewed as a *synchronous* regular language over a padded product alphabet (see below).

The resulting "diagram" is the package of automata: a *word-acceptor* for L and *multiplier automata* for R_a.

Synchronous padding (the input format for multiplier diagrams)

Let $\$ \notin A$ be a padding symbol and let $\widehat{A} := A \cup \{\$\}$. Define a padding map

$$\mathrm{pad} : A^* \times A^* \longrightarrow (\widehat{A} \times \widehat{A})^*$$

by aligning two words letter-by-letter and padding the shorter one with \$. Formally, if $u = u_1 \cdots u_m$ and $v = v_1 \cdots v_n$ and $\ell = \max\{m, n\}$, then

$$\mathrm{pad}(u, v)_i = \begin{cases} (u_i, v_i) & i \le m,\ i \le n, \\ (u_i, \$) & i \le m,\ i > n, \\ (\$, v_i) & i > m,\ i \le n. \end{cases}$$

We identify R_a with the language $\{\mathrm{pad}(u, v) : (u, v) \in R_a\} \subseteq (\widehat{A} \times \widehat{A})^*$. A *multiplier diagram* is the directed labeled graph of a finite automaton over alphabet $(\widehat{A} \times \widehat{A})$ accepting this padded language.

Existence/uniqueness (precise statement)

Theorem 17.1 (Automatic structures: invariance and what is (not) canonical)**.** *Let G be a finitely generated group.*

1. *(**Existence as a property**) The statement "G is automatic" means: for some (equivalently any) finite symmetric generating set A, there exists an automatic structure $(L, \{R_a\}_{a \in A \cup \{1\}})$ for (G, A).*
2. *(**Non-uniqueness of structure**) Even for fixed (G, A), the choice of L and the multiplier automata is generally* not *unique. Different regular combings can encode the same group.*
3. *(**Uniqueness of recognized relations**) Once (G, A) and L are fixed, each relation $R_a \subseteq L \times L$ is determined uniquely as a set. Hence the* minimal *deterministic finite automaton recognizing the padded language of R_a is unique up to isomorphism of labeled directed graphs.*
4. *(**Reconstruction of multiplication on normal forms**) If L gives* unique *normal forms (i.e. $\pi|_L$ is bijective), then the family $\{R_a\}$ uniquely determines the right-multiplication action of A on G written in normal form.*

Algorithm A (Forward): build the automatic-group diagram

Input. A finitely generated group $G = \langle A \rangle$ together with:

- a specification of a regular combing language $L \subseteq A^*$ (ideally by an automaton);
- a method to decide/compute, for each $u \in L$ and $a \in A \cup \{1\}$, some $v \in L$ with $\pi(u)a = \pi(v)$ (e.g. a known normal-form procedure, or a known automatic structure from theory).

Output. A *word-acceptor* automaton for L and multiplier automata for the padded relations R_a.

Steps (conceptual).

1. Construct (or record) a DFA/NFA $\mathcal{A}_L$ recognizing L.
2. For each $a \in A \cup \{1\}$, define the relation $R_a = \{(u, v) \in L \times L : \pi(u)a = \pi(v)\}$.
3. Build a finite automaton $\mathcal{M}_a$ recognizing the padded language $\{\mathrm{pad}(u, v) : (u, v) \in R_a\} \subseteq (\widehat{A} \times \widehat{A})^*$.
4. Output the package $(\mathcal{A}_L, \{\mathcal{M}_a\}_{a \in A \cup \{1\}})$ as the automatic-group diagram.

Boundary cases and implementation notes.

- **Surjectivity vs. uniqueness.** The definition only requires $L \twoheadrightarrow G$. If you enforce uniqueness (normal forms), Algorithm B becomes cleaner but may require extra work to compute $v \in L$ uniquely.
- **Padding symbol.** The padding symbol \$ is *not* a generator; it exists only to align pairs (u, v).
- **Identity multiplier.** The case $a = 1$ should recognize equality within L: $R_1 = \{(u, u) : u \in L\}$.

Complexity (rough). Let N_L be the number of states of the automaton for L and let N_a be the number of states in $\mathcal{M}_a$.

- **Membership in L.** Testing $w \in L$ by running $\mathcal{A}_L$ takes $O(|w|)$ time.
- **Checking a multiplier relation.** Given $u, v \in L$, testing whether $(u, v) \in R_a$ by running $\mathcal{M}_a$ on $\mathrm{pad}(u, v)$ takes $O(\max\{|u|, |v|\})$ time.
- **Building automata.** If $\mathcal{A}_L$ and $\mathcal{M}_a$ are given explicitly, producing the diagram is linear in their descriptions (e.g. $O(N_L + \sum_a N_a)$ to store). Constructing $\mathcal{M}_a$ *from scratch* depends on the chosen normal-form machinery and is the main nontrivial cost in practice.

Algorithm B (Inverse / use): solve the word problem using the automatic diagram

Input. A word-acceptor $\mathcal{A}_L$ for L and multiplier automata $\{\mathcal{M}_a\}_{a \in A \cup \{1\}}$ for an automatic structure on (G, A).

Output. A procedure to decide whether two input words represent the same element of G (word problem); optionally, a normal-form representative in L when L is unique.

Steps (standard "multiplier chase").

1. (Optional) If inputs are already in L, skip to Step 3. Otherwise choose representatives in L for the inputs. When L is a unique normal form language, compute/approximate the L-representative by multiplying the identity normal form by letters of the input word and repeatedly applying multipliers (see Step 3).
2. Given words $w_1, w_2 \in A^*$, reduce the problem to comparing elements in L: find $u, v \in L$ with $\pi(u) = \pi(w_1)$ and $\pi(v) = \pi(w_2)$.
3. To multiply an L-word u by a generator $a \in A$, find $v \in L$ with $(u, v) \in R_a$. Operationally, this means: search for an accepting run of $\mathcal{M}_a$ on some padded pair (u, v). (If L is unique and $\mathcal{M}_a$ is deterministic and functional, this v is unique.)
4. Decide whether $w_1 =_G w_2$ by checking whether the obtained u and v are equal as words in L (or whether $(u, v) \in R_1$).

Complexity (rough).

- If inputs are already in L, checking equality via R_1 is $O(|u|)$.
- Converting an arbitrary word of length n to an L-representative by iterated multiplication uses n multiplier steps. In the best-case "functional deterministic" setting, this is often near $O(n)$ in practice; in the worst case, maintaining and updating representatives can lead to $O(n^2)$ total time (depending on the structure and bookkeeping).

Worked example: $G = \mathbb{Z}$ with a regular combing and one multiplier

Take $A = \{t, t^{-1}\}$ and the regular language of *signed normal forms*

$$L \;=\; \{t^n : n \geq 0\} \;\cup\; \{(t^{-1})^n : n \geq 1\} \;\cup\; \{\varepsilon\}.$$

(So 0 is represented by ε; positives by t^n; negatives by $(t^{-1})^n$.)

Word-acceptor (for L). It accepts ε, any string of only t's, or any string of only t^{-1}'s, but rejects mixed-sign words.

Multiplier diagram for right-multiplication by t (sketch). The relation R_t consists of pairs $(u, v) \in L \times L$ with $\pi(u)t = \pi(v)$. Concretely:

- if $u = t^k$ then $v = t^{k+1}$;
- if $u = (t^{-1})$ then $v = \varepsilon$;
- if $u = (t^{-1})^k$ with $k \geq 2$ then $v = (t^{-1})^{k-1}$.

A finite automaton over the padded alphabet $(\widehat{A} \times \widehat{A})$ can recognize these cases. (For a fully explicit minimal diagram, you would list its states and transitions case-by-case; this is where you insert your preferred level of detail.)

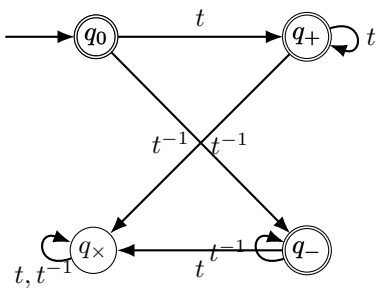

Figure 20: A schematic word-acceptor for $L \subseteq \{t, t^{-1}\}^*$. (Here double circles indicate accepting states.)

18 Automaton group automaton (Mealy automata and tree actions)

Definition

A (deterministic) *Mealy automaton* is a quadruple

$$\mathcal{M} = (Q, \Sigma, \delta, \lambda),$$

where Q is a finite set of states, Σ is a finite input alphabet,

$$\delta : Q \times \Sigma \to Q \quad \text{(transition)}, \qquad \lambda : Q \times \Sigma \to \Sigma \quad \text{(output)}.$$

We draw an edge

$$q \xrightarrow{a|b} \delta(q,a) \qquad \text{whenever } \lambda(q,a) = b.$$

The automaton is *invertible* if for every $q \in Q$ the map $a \mapsto \lambda(q,a)$ is a permutation of Σ.

Existence/uniqueness (precise statement)

Theorem 18.1 (From an invertible Mealy automaton to a group of tree automorphisms)**.** *Let $\mathcal{M} = (Q, \Sigma, \delta, \lambda)$ be a Mealy automaton.*

1. *(**Existence**) Each state $q \in Q$ defines a unique length-preserving map $f_q : \Sigma^* \to \Sigma^*$ by the recursion*
$$f_q(\varepsilon) = \varepsilon, \qquad f_q(aw) = \lambda(q,a)\, f_{\delta(q,a)}(w) \quad (a \in \Sigma,\ w \in \Sigma^*).$$
2. *(**Uniqueness**) The action f_q is uniquely determined by the labeled directed graph of $\mathcal{M}$ (i.e. by the transition/output data).*
3. *(**Group case**) If $\mathcal{M}$ is invertible, then each f_q is a rooted tree automorphism of the regular rooted tree T_Σ (with vertex set Σ^*), hence $f_q \in \mathrm{Aut}(T_\Sigma)$, and the subgroup*
$$G(\mathcal{M}) := \langle f_q : q \in Q \rangle \leq \mathrm{Aut}(T_\Sigma)$$
is called the automaton group *generated by $\mathcal{M}$.*
4. *(**Reconstruction criterion**) Conversely, if you are given a set of tree automorphisms described by wreath recursions whose set of* sections *is finite and closed under taking sections, then one can reconstruct an equivalent finite Mealy automaton producing the same action (Algorithm B below).*

Algorithm A (Forward): build the automaton-group diagram and induced action

Input. An invertible Mealy automaton $\mathcal{M} = (Q, \Sigma, \delta, \lambda)$.

Output. The Mealy diagram (state graph with labels $a \mid b$), and an oracle for the induced maps $f_q : \Sigma^* \to \Sigma^*$.

Steps.

1. **Verify invertibility.** For each $q \in Q$, check that $a \mapsto \lambda(q, a)$ is a permutation of Σ.
2. **Record the diagram.** Draw the directed labeled graph with edges $q \xrightarrow{a|\lambda(q,a)} \delta(q, a)$.
3. **Define the action.** For any word $w \in \Sigma^*$ and state $q \in Q$, compute $f_q(w)$ by the recursion $f_q(aw) = \lambda(q, a)\, f_{\delta(q,a)}(w)$.
4. **Output the group.** The automaton group is $G(\mathcal{M}) = \langle f_q : q \in Q \rangle$ (given by generators-as-states).

Boundary cases and implementation notes.

- **Non-invertible automata.** If invertibility fails, each f_q is still a tree *endomorphism* but may not be invertible, so you do not obtain a subgroup of $\mathrm{Aut}(T_\Sigma)$.
- **Minimization.** As with DFAs, Mealy automata can often be minimized (merge equivalent states), changing the diagram but not the induced action.

Complexity (rough). Let $m = |Q|$ and $k = |\Sigma|$.

- **Invertibility test.** $O(mk)$ time by scanning outputs per state (e.g. counting occurrences).
- **Apply a state to an input word.** Computing $f_q(w)$ for $|w| = n$ takes $O(n)$ time.
- **Level-n Schreier graph (optional).** The action on Σ^n has k^n vertices. Building the labeled Schreier graph for r chosen generators costs $O(r \cdot k^n \cdot n)$ time if you compute each image by $O(n)$ recursion.

Algorithm B (Inverse): reconstruct a Mealy automaton from finite wreath recursion

Input. A finite alphabet Σ and a finite set of generators acting on T_Σ given in *wreath recursion* form:

$$g \equiv (g|_a)_{a \in \Sigma}\ \pi_g,$$

where $\pi_g \in \mathrm{Sym}(\Sigma)$ is the first-letter permutation and each $g|_a$ (the *section at* a) is again one of finitely many states. Assume the set of all states you see is finite and closed under taking sections.

Output. A finite invertible Mealy automaton $\mathcal{M} = (Q, \Sigma, \delta, \lambda)$ producing the same tree action.

Steps.

1. **State set.** Let Q be the finite closure of the given generators under taking sections $g \mapsto g|_a$ for all $a \in \Sigma$.
2. **Output function.** For each $q \in Q$ and $a \in \Sigma$, define $\lambda(q, a) := \pi_q(a)$.
3. **Transition function.** Define $\delta(q, a) := q|_a$.
4. **Return the automaton.** Output $\mathcal{M} = (Q, \Sigma, \delta, \lambda)$.

Boundary cases.

- **Infinite-state closure.** If the closure under sections is infinite, then no finite Mealy automaton can encode the action with finitely many states.
- **Non-invertible recursion.** If some π_q is not a permutation, the resulting machine is a Mealy automaton but not invertible.

Complexity (rough). If the closure process yields $|Q| = m$ states, then constructing δ, λ takes $O(mk)$ time and space.

Worked example: the binary adding machine (odometer)

Let $\Sigma = \{0, 1\}$. Consider two states I (identity) and A (add 1 with carry):

$$I: \ 0 \mid 0 \to I, \ \ 1 \mid 1 \to I; \qquad A: \ 0 \mid 1 \to I, \ \ 1 \mid 0 \to A.$$

This Mealy automaton is invertible (each state outputs a permutation of $\{0, 1\}$) and defines an automaton group $\langle f_A \rangle \cong \mathbb{Z}$.

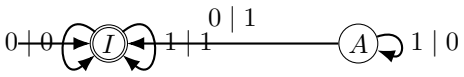

Figure 21: Binary adding machine: A adds 1 (least significant digit first, i.e. acting on the rooted binary tree).

Forward construction (Algorithm A) in one check. Applying A to a word $w \in \{0, 1\}^*$ toggles leading 1's to 0 until the first 0 becomes 1 (carry mechanism), which is exactly adding one in binary on each finite level Σ^n.

Inverse reconstruction (Algorithm B) from wreath recursion. State A has first-letter permutation (0 1) and sections $(A|_0, A|_1) = (I, A)$, hence

$$A \equiv (I, A)\,(0\ 1), \qquad I \equiv (I, I)\,\mathrm{id},$$

which yields λ, δ as above.

References

[1] M. R. Bridson and A. Haefliger. *Metric Spaces of Non-Positive Curvature*, volume 319 of *Grundlehren der mathematischen Wissenschaften*. Springer, 1999.

[2] A. Cayley. Desiderata and suggestions: No. 2. the theory of groups: Graphical representation. *American Journal of Mathematics*, 1(2):174–176, 1878.

[3] R. Charney. An introduction to right-angled artin groups. *Geometriae Dedicata*, 125:141–158, 2007.

[4] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein. *Introduction to Algorithms*. MIT Press, 3 edition, 2009.

[5] M. Dehn. Über unendliche diskontinuierliche gruppen. *Mathematische Annalen*, 71(1):116–144, 1911.

[6] S. M. Gersten. Reducible diagrams and equations over groups. In S. M. Gersten, editor, *Essays in Group Theory*, volume 8 of *Mathematical Sciences Research Institute Publications*, pages 15–73. Springer, New York, 1987.

[7] E. R. Green. *Graph Products of Groups*. PhD thesis, University of Leeds, 1990.

[8] M. Greendlinger. Dehn's algorithm for the word problem. *Communications on Pure and Applied Mathematics*, 13(1):67–83, 1960.

[9] M. Gromov. Hyperbolic groups. In S. M. Gersten, editor, *Essays in Group Theory*, volume 8 of *Mathematical Sciences Research Institute Publications*, pages 75–263. Springer, New York, 1987.

[10] A. Hatcher. *Algebraic Topology*. Cambridge University Press, 2002.

[11] D. F. Holt, B. Eick, and E. A. O'Brien. *Handbook of Computational Group Theory*. Chapman and Hall/CRC, 2005.

[12] J. Howie. On pairs of 2-complexes and systems of equations over groups. *Journal für die reine und angewandte Mathematik*, 324:165–174, 1981.

[13] J. Howie. Spherical diagrams and equations over groups. *Mathematical Proceedings of the Cambridge Philosophical Society*, 96(2):255–268, 1984.

[14] I. Kapovich and A. Myasnikov. Stallings foldings and the subgroup structure of free groups. *Journal of Algebra*, 248(2):608–668, 2002.

[15] R. C. Lyndon and P. E. Schupp. *Combinatorial Group Theory*, volume 89 of *Ergebnisse der Mathematik und ihrer Grenzgebiete*. Springer, Berlin–New York, 1977.

[16] M. Salvetti. Topology of the complement of real hyperplanes in $\mathbb{C}^N$. *Inventiones Mathematicae*, 88:603–618, 1987.

[17] O. Schreier. Die untergruppen der freien gruppen. *Abhandlungen aus dem Mathematischen Seminar der Universität Hamburg*, 5(1):161–183, 1927.

[18] J.-P. Serre. *Trees*. Springer, Berlin–New York, 1980. Translated from the French by John Stillwell.

[19] A. J. Sieradski. A coloring test for asphericity. *The Quarterly Journal of Mathematics*, 34(1):97–106, 1983.

[20] J. R. Stallings. Topology of finite graphs. *Inventiones Mathematicae*, 71(3):551–565, 1983.

[21] R. E. Tarjan. Efficiency of a good but not linear set union algorithm. *Journal of the ACM*, 22(2):215–225, 1975.

[22] J. A. Todd and H. S. M. Coxeter. A practical method for enumerating cosets of a finite abstract group. *Proceedings of the Edinburgh Mathematical Society (Series 2)*, 5(1):26–34, 1936.

[23] N. W. M. Touikan. A fast algorithm for stallings' folding process. *International Journal of Algebra and Computation*, 16(6):1031–1045, 2006.

[24] E. R. van Kampen. On some lemmas in the theory of groups. *American Journal of Mathematics*, 55(1):268–273, 1933.